

Université Hassan Premier
Faculté des Sciences et Techniques de Settat
Département d'Informatique
Filière : Génie Informatique
Année universitaire 2025–2026



RAPPORT DE PROJET DE FIN DE MODULE

Centralized AI Monitoring Platform

*Plateforme centralisée de supervision d'infrastructure
assistée par intelligence artificielle*

Réalisé par :

CHAHIDY Rim
SAMDI Wiam
TAÏK Youssef

Encadré par :

Mr. Ezzati

Résumé

Résumé — Français

La supervision des infrastructures informatiques constitue aujourd'hui un enjeu stratégique pour toute organisation exploitant des serveurs. Les outils traditionnels de surveillance tels que Prometheus, Grafana ou Datadog se concentrent sur l'affichage de métriques, laissant aux ingénieurs la charge d'interpréter les données et de diagnostiquer manuellement les incidents. Cette approche génère une *fatigue d'alertes* importante et allonge considérablement le temps moyen de résolution (MTTR).

Ce rapport présente **caimp** (*Centralized AI Monitoring Platform*), une plateforme open source auto-hébergée qui repositionne la supervision autour d'une philosophie *réponses-en-premier* : au lieu d'afficher des graphes, le tableau de bord répond directement à la question « Quelque chose est-il cassé, et si oui, pourquoi ? »

L'architecture repose sur 19 microservices conteneurisés communiquant via NATS JetStream. Un agent léger déployé sur chaque serveur cible collecte les métriques système via OTLP et signale automatiquement les changements d'environnement (déploiements Git, événements Docker, installations de paquets, connexions SSH). Un moteur de corrélation associe chaque anomalie détectée aux modifications survenues dans les 30 minutes précédant l'incident et soumet l'analyse à un LLM local (Llama 3.1 8B via Ollama) pour produire une explication en langage naturel.

Le moteur de prévision applique le lissage exponentiel de Holt-Winters sur les 24 heures à venir. Un moteur de déduplication d'alertes élimine les doublons sur une fenêtre de 15 minutes. La plateforme est installable en moins de 10 minutes via Docker Compose.

Mots-clés : AIOps, supervision d'infrastructure, microservices, grand modèle de langage, TimescaleDB, NATS JetStream, corrélation de changements, détection d'anomalies, Holt-Winters, RAG, pgvector.

Abstract

Abstract — English

Infrastructure monitoring is a strategic challenge for any organisation operating servers. Traditional tools such as Prometheus, Grafana or Datadog focus on displaying metrics, leaving engineers to manually diagnose incidents. This generates significant *alert fatigue* and increases the mean time to resolution (MTTR).

This report presents **caimp** (*Centralized AI Monitoring Platform*), a self-hosted open-source platform built around an *answers-first* philosophy : instead of graphs, the dashboard answers directly “*Is anything broken, and if so, why ?*”

The architecture uses 19 containerised microservices communicating via NATS JetStream. A lightweight agent collects metrics via OTLP and automatically reports system changes. A change correlation engine scores each anomaly against changes in the 30-minute window before it, then calls a local LLM (Llama 3.1 8B via Ollama) for a plain-English explanation and recommended action.

A Holt-Winters forecasting engine predicts resource saturation over 24 hours. An alert deduplication engine eliminates duplicates in a 15-minute window. The platform installs in under 10 minutes via Docker Compose.

Keywords : AIOps, infrastructure monitoring, microservices, large language model, TimescaleDB, NATS JetStream, change correlation, anomaly detection, Holt-Winters, RAG, pgvector.

Table des matières

Résumé	i
Abstract (English)	ii
Table des matières	vii
Liste des figures	viii
Liste des tableaux	ix
Liste des abréviations	x
1. Introduction générale	1
1.1 Contexte et motivation	1
1.2 Objectifs du projet	2
1.3 Méthodologie de travail	2
1.4 Plan du rapport	3
2. Contexte général et cadre du projet	4
2.1 Observabilité moderne des systèmes informatiques	4
2.1.1 Les trois piliers de l'observabilité	4
2.1.2 De la supervision réactive à la supervision prédictive	5
2.2 Le domaine de l'AIOps	5
2.2.1 Définition et périmètre	5
2.2.2 Évolution historique	5
2.3 Problématique	6
2.3.1 La fatigue d'alertes : un phénomène documenté	6
2.3.2 Le fossé entre détection et compréhension	6
2.3.3 Le problème spécifique des petites équipes	6
2.3.4 Formulation de la problématique	7
2.4 Périmètre fonctionnel	7
2.4.1 Ce qui est inclus	7
2.4.2 Ce qui est exclu	7
3. État de l'art	9
3.1 Solutions de supervision commerciales	9
3.1.1 Datadog	9
3.1.2 Grafana + Prometheus	9

3.1.3	New Relic	10
3.1.4	Dynatrace	10
3.1.5	Splunk + ITSI	10
3.1.6	Alternatives open source : SigNoz, Netdata, Zabbix, Nagios	10
3.2	Tableau comparatif	11
3.3	Techniques de détection d'anomalies	11
3.3.1	Détection par seuil statique	11
3.3.2	Score Z et détection statistique	11
3.3.3	Détection par taux de variation	11
3.3.4	Forêt d'isolation (Isolation Forest)	12
3.3.5	Lissage exponentiel de Holt-Winters	12
3.4	Modèles de langage pour l'AIOPS	12
3.4.1	Panorama des LLM ouverts	12
3.4.2	Quantification des modèles	13
3.4.3	Le pattern RAG (Retrieval-Augmented Generation)	13
3.5	Positionnement de CAIMP	13
4.	Analyse et spécification des besoins	15
4.1	Identification des acteurs	15
4.2	Besoins fonctionnels	15
4.3	Besoins non fonctionnels	17
4.3.1	Performance	17
4.3.2	Sécurité	17
4.3.3	Disponibilité et résilience	17
4.3.4	Maintenabilité	18
4.4	Diagramme de cas d'utilisation global	18
4.5	Diagrammes de séquence	18
4.5.1	Enrôlement d'un agent	19
4.5.2	Détection d'anomalie et explication IA	20
4.6	Matrice de traçabilité des besoins	20
5.	Conception et architecture	22
5.1	Choix architectural : microservices vs. monolithe	22
5.2	Architecture globale	23
5.3	Fonctionnement de l'architecture — flux de bout en bout	25
5.3.1	Flux 1 — Collecte et ingestion des métriques	25
5.3.2	Flux 2 — Analyse IA et explication de l'anomalie	25
5.3.3	Flux 3 — Prévision des ressources	26
5.3.4	Flux 4 — Mise à jour en temps réel du navigateur	26
5.3.5	Flux 5 — Requête utilisateur depuis le tableau de bord	26
5.4	Bus d'événements : NATS JetStream	27
5.4.1	Choix du bus de messages	27

5.4.2	Streams et sujets	27
5.5	Modèle de données	28
5.5.1	Principes de conception	28
5.5.2	Schéma relationnel	28
5.5.3	Diagramme entité-association	30
5.6	Flux de traitement détaillés	30
5.6.1	Flux 1 : Ingestion de métriques	30
5.6.2	Flux 2 : Corrélation de changements	30
5.7	Sécurité : modèle de menaces STRIDE	31
6.	Étude des technologies utilisées	32
6.1	Langages de programmation	32
6.1.1	Python 3.12	32
6.1.2	Go 1.22	32
6.1.3	TypeScript (React 18)	33
6.2	Frameworks backend	33
6.2.1	FastAPI	33
6.2.2	asynpg	33
6.3	Base de données	33
6.3.1	PostgreSQL 16 + TimescaleDB	33
6.3.2	pgvector	34
6.3.3	Redis 7.2	34
6.4	Intelligence artificielle	34
6.4.1	Ollama	34
6.4.2	Holt-Winters — Formulation mathématique complète	35
6.5	Infrastructure	35
6.5.1	Docker et Docker Compose v2	35
6.5.2	Nginx	35
6.5.3	Protocole OTLP	36
6.6	Sécurité	36
6.6.1	JWT (JSON Web Token)	36
6.6.2	mTLS pour les agents	36
7.	Réalisation et implémentation	37
7.1	Organisation du dépôt	37
7.2	Répartition des tâches	38
7.3	Module 1 : Agent de supervision (Python)	38
7.3.1	Collecte de métriques	38
7.3.2	Enrôlement mTLS	39
7.4	Module 2 : Telemetry Writer (Go)	39
7.4.1	Détecteur d'anomalies	39
7.5	Module 3 : AI Worker et pipeline RAG	40

7.5.1	Construction du prompt	40
7.6	Module 4 : Moteur de prévision (Holt-Winters)	41
7.6.1	Implémentation NumPy	41
7.7	Module 5 : Moteur de corrélation de changements	42
7.8	Module 6 : Frontend React	42
7.8.1	Tableau de bord orienté réponses	42
7.9	Module 7 : Génération de rapports PDF	43
7.10	Difficultés rencontrées et solutions	44
8.	Tests et validation	46
8.1	Stratégie de tests : la pyramide	46
8.2	Tests unitaires	46
8.2.1	Tests des détecteurs d'anomalies	46
8.2.2	Tests du moteur Holt-Winters	47
8.3	Tests d'intégration	48
8.4	Tests de charge	48
8.5	Tests de sécurité	49
8.6	Matrice de traçabilité besoins – tests	49
9.	Déploiement et mise en production	51
9.1	Modes de déploiement	51
9.2	Procédure d'installation complète	51
9.2.1	Prérequis	51
9.2.2	Étapes d'installation	51
9.2.3	Configuration des variables d'environnement	52
9.3	Mode Low-RAM (4 Go)	53
9.4	Sauvegarde et restauration	53
9.5	Monitoring de la plateforme elle-même	53
9.6	Génération de métriques de démonstration	53
9.7	Surveillance d'une machine virtuelle réelle	54
9.8	Accès aux tableaux de bord	55
10.	Résultats, discussion et perspectives	57
10.1	Résultats obtenus	57
10.1.1	Fonctionnalités livrées	57
10.1.2	Métriques de la base de code	58
10.2	Forces du projet	58
10.3	Limites identifiées	59
10.4	Perspectives d'évolution	59
10.4.1	Court terme (3–6 mois)	59
10.4.2	Moyen terme (6–18 mois)	59
10.4.3	Vision long terme	60

11. Conclusion générale	61
11.1 Synthèse du travail réalisé	61
11.2 Apports académiques et professionnels	61
11.3 Bilan personnel	62
11.4 Mot de la fin	62
Références bibliographiques	64
A. Schéma de la base de données	66
B. Référence des endpoints API	69
B.1 Admin API (port 8001)	69
B.2 Query API (port 8002)	69
B.3 AI Orchestrator (port 8010)	70
B.4 Change Tracker (port 8011)	70
C. Extraits du fichier docker-compose.yml	71
D. Glossaire technique	73

Table des figures

4.1	Diagramme de cas d'utilisation global de CAIMP	18
4.2	Séquence d'enrôlement d'un agent de supervision	19
4.3	Séquence de détection d'anomalie et génération d'explication IA	20
5.1	Architecture globale de la plateforme CAIMP — 7 couches, 19 services	24
5.2	Diagramme entité-association simplifié de CAIMP (17 tables)	30
8.1	Pyramide de tests de CAIMP	46

Liste des tableaux

3.1	Comparaison des solutions de supervision selon 12 critères	11
4.1	Extrait de la matrice de traçabilité besoins fonctionnels – composants . . .	20
5.1	Comparaison monolithe vs. microservices pour CAIMP	22
5.2	Comparaison des solutions de messagerie	27
5.3	Streams NATS JetStream de CAIMP	28
5.4	Analyse STRIDE et contre-mesures de CAIMP	31
7.1	Répartition des responsabilités entre les membres de l'équipe	38
7.2	Principales difficultés rencontrées et solutions apportées	44
8.1	Résultats du test de charge à 1 000 métriques/seconde	49
8.2	Tests de sécurité réalisés	49
8.3	Matrice de traçabilité besoins fonctionnels – couverture par tests	49
9.1	Modes de déploiement et ressources requises	51
9.2	Variables d'environnement obligatoires	52
9.3	Variables d'environnement de l'agent de démonstration	54
9.4	URLs des interfaces web	55
10.1	Matrice objectifs initiaux – résultats obtenus	57
10.2	Indicateurs quantitatifs de la base de code CAIMP	58

Liste des abréviations

Sigle	Signification
AI	<i>Artificial Intelligence</i> — Intelligence Artificielle
AIOps	<i>AI for IT Operations</i> — Application de l'IA aux opérations informatiques
API	<i>Application Programming Interface</i> — Interface de programmation
CA	<i>Certificate Authority</i> — Autorité de certification
CAIMP	<i>Centralized AI Monitoring Platform</i> — Plateforme centralisée de supervision par IA
CI/CD	<i>Continuous Integration / Continuous Deployment</i>
CPU	<i>Central Processing Unit</i> — Processeur central
CNCF	<i>Cloud Native Computing Foundation</i>
DB	<i>Database</i> — Base de données
DevOps	<i>Development + Operations</i>
DNS	<i>Domain Name System</i>
ERD	<i>Entity-Relationship Diagram</i> — Diagramme entité-association
FST	Faculté des Sciences et Techniques
FS	<i>File System</i> — Système de fichiers
GDPR	<i>General Data Protection Regulation</i>
gRPC	<i>Google Remote Procedure Call</i>
HEC	<i>HTTP Event Collector</i> (Splunk)
HNSW	<i>Hierarchical Navigable Small World</i> — index vectoriel
HTTP	<i>HyperText Transfer Protocol</i>
IA	Intelligence Artificielle
IVFFlat	<i>Inverted File Flat</i> — index vectoriel approximatif
JSON	<i>JavaScript Object Notation</i>
JWT	<i>JSON Web Token</i>
KPI	<i>Key Performance Indicator</i> — Indicateur de performance clé
LLM	<i>Large Language Model</i> — Grand modèle de langage
MAE	<i>Mean Absolute Error</i> — Erreur absolue moyenne

Sigle	Signification
MLTK	<i>Machine Learning Toolkit</i> (Splunk)
MTTR	<i>Mean Time To Resolution</i> — Temps moyen de résolution
mTLS	<i>mutual Transport Layer Security</i> — TLS mutuel
NATS	<i>Neural Autonomic Transport System</i>
NOC	<i>Network Operations Center</i>
OTLP	<i>OpenTelemetry Protocol</i>
PDF	<i>Portable Document Format</i>
PKI	<i>Public Key Infrastructure</i>
Python	Langage de programmation Python (version 3.12)
RAG	<i>Retrieval-Augmented Generation</i>
RAM	<i>Random Access Memory</i> — Mémoire vive
RBAC	<i>Role-Based Access Control</i> — Contrôle d'accès par rôle
RCA	<i>Root Cause Analysis</i> — Analyse de la cause racine
REST	<i>Representational State Transfer</i>
RLS	<i>Row-Level Security</i> — Sécurité au niveau de la ligne
SaaS	<i>Software as a Service</i>
SLA	<i>Service Level Agreement</i> — Accord de niveau de service
SPL	<i>Search Processing Language</i> (Splunk)
SQL	<i>Structured Query Language</i>
SRE	<i>Site Reliability Engineering</i>
SSH	<i>Secure Shell</i>
SSE	<i>Server-Sent Events</i>
STRIDE	<i>Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege</i>
TLS	<i>Transport Layer Security</i>
TSD	<i>Time-Series Database</i> — Base de données temporelles
TS	TypeScript
UHP	Université Hassan Premier
UI	<i>User Interface</i> — Interface utilisateur
UML	<i>Unified Modeling Language</i>
UUID	<i>Universally Unique Identifier</i>
VPS	<i>Virtual Private Server</i>
WS	<i>WebSocket</i>
YAML	<i>YAML Ain't Markup Language</i>

Chapitre 1

Introduction générale

Introduction

L'essor fulgurant des architectures *cloud-native* au cours de la dernière décennie a profondément transformé la manière dont les organisations déploient et gèrent leurs systèmes informatiques. Les infrastructures modernes, composées de dizaines voire de centaines de serveurs, de conteneurs et de services distribués, génèrent des flux de données opérationnelles d'une densité sans précédent. Dans ce contexte, la supervision des systèmes — autrefois déléguée à un administrateur réseau scrutant quelques courbes — est devenue une discipline à part entière, au cœur de la résilience organisationnelle.

Pourtant, les outils actuels de surveillance, aussi sophistiqués soient-ils, partagent une limite fondamentale : ils savent *détecter* qu'un problème existe, mais peinent à expliquer *pourquoi* il est apparu. Face à une alerte « CPU à 98 % », l'ingénieur de garde doit encore mener une investigation manuelle qui peut durer de trente minutes à plusieurs heures, consultant successivement les journaux système, l'historique des déploiements, les requêtes base de données et les métriques réseau. Ce gouffre entre *détection* et *compréhension* constitue précisément le problème que ce projet cherche à résoudre.

1.1 Contexte et motivation

Les statistiques sectorielles confirment l'ampleur du défi. Selon le rapport *State of Digital Operations* de PagerDuty (2023) [1], le coût moyen d'une heure d'indisponibilité s'élève à plus de 300 000 dollars pour les grandes entreprises, tandis que le temps moyen de résolution (MTTR) dépasse quarante-cinq minutes dans les organisations ne disposant pas de pratiques AIOps matures.

Pour les petites équipes — une startup avec deux développeurs gérant un serveur VPS, une agence web administrant une dizaine de machines — la situation est encore plus critique. Ces équipes n'ont ni la ressource humaine d'un ingénieur *DevOps* dédié, ni le budget pour des solutions commerciales à plusieurs milliers d'euros par mois. Elles ont besoin d'un outil qui réponde simplement à la question : « *Mon serveur est-il en bonne santé ? Sinon, que dois-je faire ?* »

C'est dans ce contexte que s'inscrit le projet **caimp** — *Centralized AI Monitoring Platform* — une plateforme auto-hébergée et *open source* qui positionne l'intelligence artificielle non pas comme un gadget d'affichage, mais comme le cœur du processus de diagnostic.

1.2 Objectifs du projet

Les objectifs du projet CAIMP ont été définis de manière itérative en tenant compte des lacunes identifiées dans les solutions existantes :

- O1. Tableau de bord orienté réponses** : concevoir une interface dont la page d'accueil présente immédiatement le statut global, les problèmes potentiels et les actions recommandées — sans nécessiter d'expertise en monitoring de la part de l'utilisateur.
- O2. Corrélation automatique des changements** : développer un moteur capable d'associer chaque anomalie détectée aux modifications récentes de l'environnement (déploiements, redémarrages, mises à jour de paquets, connexions SSH) afin d'automatiser la recherche de la cause racine.
- O3. Explication en langage naturel** : intégrer un modèle de langage local (sans appel à un API cloud) pour produire une explication compréhensible par un développeur sans formation *DevOps*.
- O4. Prévision des saturations** : implémenter un moteur de prévision basé sur le lissage exponentiel de Holt-Winters pour anticiper les dépassements de seuils dans les 24 heures à venir.
- O5. Architecture microservices robuste** : concevoir une architecture découplée, basée sur un bus d'événements (NATS JetStream), permettant la scalabilité horizontale et l'isolation multi-tenant des données.
- O6. Sécurité de bout en bout** : implémenter l'authentification mutuelle TLS pour les agents, la gestion de jetons JWT, et l'isolation des données par organisation via des politiques RLS en base de données.
- O7. Déploiement accessible** : rendre l'installation complète réalisable en moins de dix minutes via Docker Compose, sur n'importe quelle machine disposant de 6 Go de RAM.

1.3 Méthodologie de travail

Le développement de CAIMP a suivi une approche *agile* adaptée au contexte académique. L'équipe de trois personnes s'est réunie quotidiennement pour des sessions de synchronisation courtes (*stand-ups*), et a organisé son travail en sprints hebdomadaires d'une durée de cinq jours.

Le dépôt de code a été hébergé sur GitHub, avec une branche principale protégée

(**main**) et une branche de développement (**dev**) depuis laquelle les fonctionnalités ont été développées et testées avant intégration. Les messages de *commit* ont suivi la convention *Conventional Commits* (**feat:**, **fix:**, **chore:**, **docs:**), facilitant la génération automatique du journal des modifications.

La répartition des responsabilités a été organisée par domaine d'expertise : le *backend* (services Python, Go, base de données) a mobilisé les trois membres en rotation, le *frontend* React a été principalement pris en charge par un membre spécialisé, et l'intégration des modèles LLM et du moteur de corrélation a constitué un chantier partagé en binôme.

1.4 Plan du rapport

Ce rapport est structuré en dix chapitres, organisés pour guider le lecteur depuis le contexte général jusqu'aux perspectives d'évolution :

- Le **chapitre 1** présente le contexte général du projet, les enjeux de l'observabilité moderne et le domaine de l'AIOPS.
- Le **chapitre 2** analyse l'état de l'art des solutions de supervision existantes et positionne CAIMP par rapport à elles.
- Le **chapitre 3** spécifie les besoins fonctionnels et non fonctionnels, et présente les diagrammes UML d'analyse.
- Le **chapitre 4** décrit en détail l'architecture de la plateforme, le modèle de données et les flux de traitement.
- Le **chapitre 5** justifie les choix technologiques.
- Le **chapitre 6** présente les détails d'implémentation des modules clés, avec extraits de code commentés.
- Le **chapitre 7** documente la stratégie de tests et les résultats de validation.
- Le **chapitre 8** décrit les procédures de déploiement et de mise en production.
- Le **chapitre 9** discute les résultats obtenus et les perspectives d'évolution.
- Le **chapitre 10** conclut le rapport et dresse un bilan personnel de l'expérience.

Conclusion

Ce projet représente l'aboutissement d'un mois de travail intense, pendant lequel une plateforme complète de supervision d'infrastructure a été conçue, développée, testée et documentée. Les chapitres qui suivent témoignent de la rigueur scientifique et technique que nous avons cherché à maintenir tout au long de cette démarche. Le chapitre suivant pose les bases contextuelles indispensables à la compréhension de l'ensemble du travail réalisé.

Chapitre 2

Contexte général et cadre du projet

Introduction

Ce premier chapitre pose le cadre conceptuel du projet CAIMP. Nous décrivons l'environnement technologique de l'observabilité moderne et du domaine des AIOps, puis nous formulons la problématique de manière précise en appui sur des données chiffrées issues de sources industrielles et académiques.

2.1 Observabilité moderne des systèmes informatiques

2.1.1 Les trois piliers de l'observabilité

L'observabilité désigne la capacité à inférer l'état interne d'un système à partir de ses sorties externes [2]. Dans le domaine de l'informatique distribuée, elle repose traditionnellement sur trois piliers complémentaires :

Les métriques (*metrics*)

Des mesures numériques périodiques sur l'état du système : usage du CPU, occupation mémoire, taux de requêtes, latence. Leur collecte est légère et leur analyse statistique permet la détection d'anomalies et la prévision des tendances.

Les journaux (*logs*)

Des enregistrements textuels d'événements discrets : erreurs d'application, requêtes HTTP, démarrages de service. Riches en information contextuelle, ils sont indispensables au diagnostic post mortem mais difficiles à analyser à grande échelle sans outillage adapté.

Les traces (*traces*)

Des représentations du parcours d'une requête à travers les différents composants d'un système distribué. Elles permettent de localiser les goulots d'étranglement et d'identifier les dépendances entre services [3].

Le protocole OTLP (*OpenTelemetry Protocol*) [4], standardisé par la CNCF, unifie la collecte de ces trois types de signaux sous un format commun, facilitant l'interopérabilité

entre les outils d'instrumentation et les *backends* d'analyse.

2.1.2 De la supervision réactive à la supervision prédictive

Les premières générations d'outils de monitoring, comme Nagios ou Zabbix, adoptaient une approche purement réactive : envoyer une alerte quand un seuil est dépassé. Cette philosophie présentait deux limitations majeures. D'une part, les seuils statiques génèrent une proportion significative de faux positifs et de faux négatifs sur des systèmes à charge variable. D'autre part, l'alerte ne transmet aucune information sur la *cause* du problème.

L'émergence de bases de données temporelles hautement performantes comme TimescaleDB [5] et de techniques statistiques de détection d'anomalies (*Z-score*, *Isolation Forest* [6], *Holt-Winters* [7]) a permis de passer à un paradigme proactif : détecter une dérive avant qu'elle ne devienne un incident, et anticiper les saturations futures.

2.2 Le domaine de l'AIOPS

2.2.1 Définition et périmètre

L'AIOPS (*Artificial Intelligence for IT Operations*) désigne l'application des techniques d'apprentissage automatique et d'intelligence artificielle aux opérations informatiques [8]. Gartner, qui a forgé le terme en 2017, le définit comme « une plateforme qui combine le Big Data, l'apprentissage automatique et les technologies analytiques avancées pour améliorer et automatiser directement les opérations informatiques ».

Les domaines couverts par l'AIOPS incluent :

- La corrélation automatique d'événements et d'alertes ;
- La détection d'anomalies sans règles prédéfinies (*unsupervised*) ;
- L'analyse de la cause racine (RCA) et la suggestion de remédiation ;
- La prévision de capacité et la gestion prédictive des incidents ;
- L'automatisation des actions de remédiation (*runbooks* automatisés).

2.2.2 Évolution historique

L'émergence de l'AIOPS s'inscrit dans une évolution continue des pratiques opérationnelles. Les grandes étapes sont les suivantes :

1. **Années 1990–2000 : supervision par seuils statiques** (Nagios, MRTG). Règles manuelles, alertes binaires, forte dépendance à l'expertise humaine.
2. **Années 2000–2010 : agrégation de journaux et métriques** (Splunk, Graphite, Ganglia). Centralisation des données mais analyse encore manuelle.
3. **Années 2010–2018 : observabilité distribuée** (Prometheus, Grafana, Jaeger, Zipkin). Instrumentation des architectures microservices, premières détections d'anomalies statistiques.

4. **Années 2018–2023 : aiops émergent** (Dynatrace Davis, Datadog Watchdog). Algorithmes ML intégrés dans les plateformes commerciales, corrélation automatique d'anomalies.
5. **2023–present : aiops générative** (LLM locaux, RAG). Explosion des modèles de langage ouverts permettant l'explication en langage naturel sans dépendance cloud.

2.3 Problématique

2.3.1 La fatigue d'alertes : un phénomène documenté

Les études industrielles convergent pour identifier la *fatigue d'alertes* comme l'un des défis opérationnels les plus critiques. Selon le rapport PagerDuty 2023 [1] :

- 73 % des ingénieurs déclarent ignorer régulièrement des alertes en raison de leur volume excessif ;
- 40 % des alertes sont des faux positifs ou des doublons ;
- Le MTTR moyen est de 4 heures 20 minutes pour les organisations sans outillage AIOPS, contre 42 minutes pour celles qui l'ont adopté.

2.3.2 Le fossé entre détection et compréhension

La réponse standard à une alerte « CPU à 98 % » suit un protocole d'investigation implicite que tout ingénieur d'exploitation reconnaîtra :

1. Vérifier les processus consommateurs (`top`, `htop`) ;
2. Consulter les journaux d'application (`journalctl`, `tail -f`) ;
3. Examiner l'historique des déploiements récents ;
4. Vérifier les requêtes base de données lentes ;
5. Consulter les métriques réseau pour écarter une attaque DDOS ;
6. Croiser avec les événements système (`dmesg`, `auditd`).

Cette séquence, répétée à 2h du matin lors d'une astreinte, est source d'erreurs et de stress. Elle suppose une expertise que les petites équipes n'ont pas systématiquement, et elle mobilise un temps précieux pendant lequel les utilisateurs subissent l'indisponibilité du service.

2.3.3 Le problème spécifique des petites équipes

Les solutions commerciales d'AIOPS sont conçues pour les grandes organisations. Datadog, New Relic ou Dynatrace proposent des tarifications à la donnée ou au *host* qui représentent une charge financière prohibitive pour une startup ou une agence : entre 15 et 23 \$ par hôte et par mois pour les solutions de base, pouvant atteindre plusieurs milliers de dollars mensuels pour des fonctionnalités avancées.

Par ailleurs, ces outils requièrent une expertise significative pour être correctement

configurés, et leur courbe d'apprentissage décourage les équipes sans ingénieur DEVOPS dédié.

2.3.4 Formulation de la problématique

Cette analyse nous conduit à formuler la problématique centrale du projet :

Problématique

Comment concevoir une plateforme de supervision d'infrastructure auto-hébergée, accessible à des équipes sans expertise DEVOPS, capable non seulement de détecter les anomalies mais d'en expliquer la cause en langage naturel et de proposer des actions correctives concrètes, le tout sans dépendance à des services cloud tiers ?

2.4 Périmètre fonctionnel

2.4.1 Ce qui est inclus

- Collecte de métriques CPU, mémoire et disque via un agent Python léger et le protocole OTLP ;
- Détection d'anomalies par trois algorithmes (seuil statique, Z-score, taux de variation) ;
- Analyse de la cause racine par corrélation des changements récents et appel à un LLM local ;
- Prévision sur 24 heures par lissage de Holt-Winters ;
- Interface utilisateur orientée réponses avec tableau de bord synthétique ;
- Déduplication d'alertes et détection de bursts ;
- Rapports PDF lisibles par des non-techniciens ;
- Intégration optionnelle avec Splunk Enterprise ;
- Déploiement complet par Docker Compose ;
- Authentification JWT et chiffrement MTLS pour les agents.

2.4.2 Ce qui est exclu

- Collecte de traces (*distributed tracing*) : hors périmètre, nécessiterait l'instrumentation des applications cibles ;
- Remédiation automatisée (exécution automatique de corrections) : les recommandations restent à la discrétion de l'opérateur ;
- Support Windows pour l'agent : limité à Linux dans cette version ;
- Interface mobile : la plateforme est optimisée pour navigateur de bureau ;
- Authentification fédérée (SAML, OIDC) : non implémentée dans le périmètre de ce projet.

Conclusion

Ce chapitre a défini le cadre conceptuel du projet CAIMP, en ancrant la problématique dans les enjeux réels de la supervision moderne. L'analyse de la fatigue d'alertes et du fossé entre détection et compréhension justifie l'approche orientée AIOPS adoptée. Le chapitre suivant brosse un tableau comparatif des solutions existantes et positionne CAIMP dans cet écosystème.

Chapitre 3

État de l'art

Introduction

Ce chapitre examine les solutions de supervision existantes, les techniques de détection d'anomalies disponibles, et les avancées récentes en matière de modèles de langage appliqués aux opérations informatiques. Cette analyse comparative est indispensable pour justifier les choix architecturaux de CAIMP et démontrer la valeur ajoutée de l'approche retenue.

3.1 Solutions de supervision commerciales

3.1.1 Datadog

Datadog est une plateforme SAAS de monitoring fondée en 2010, devenue l'une des références du marché avec plus de 27 000 clients en 2023. Son architecture repose sur un agent léger (**datadog-agent**) installé sur chaque hôte, qui collecte métriques, traces et journaux et les envoie vers les datacenters Datadog.

Points forts : intégrations natives avec plus de 700 technologies, tableau de bord personnalisable, fonctionnalités d'apprentissage automatique (*Watchdog*) pour la détection d'anomalies, corrélation automatique des signaux, APM (*Application Performance Monitoring*).

Limites : tarification à l'hôte (15 \$/mois par hôte pour l'offre Infrastructure, 31 \$/mois avec APM) rédhibitoire pour les petites équipes; données hébergées aux États-Unis, posant des questions de conformité RGPD; aucun déploiement *on-premise* disponible; les explications générées par l'IA restent superficielles et ne s'appuient pas sur un LLM génératif.

3.1.2 Grafana + Prometheus

La combinaison Grafana / Prometheus est devenue le standard *de facto* de l'observabilité open source. Prometheus [5] est un système de surveillance pull-based avec une base de données temporelles propriétaire (TSDB) et un langage de requête dédié (PromQL). Grafana est un outil de visualisation connectant des dizaines de sources de données.

Points forts : entièrement open source, communauté active, excellent pour les archi-

tections Kubernetes, grande flexibilité des alertes (Alertmanager), intégrations étendues.

Limites : absence totale d'IA native; TSDB de Prometheus non conçue pour les requêtes analytiques complexes (pas de fenêtres glissantes, pas d'agrégats temporels avancés); nécessite une expertise significative pour la configuration; génère facilement de la fatigue d'alertes sans déduplication intelligente.

3.1.3 New Relic

New Relic est une plateforme SAAS centrée sur la performance applicative (APM) avec des capacités d'infrastructure monitoring. Son architecture Telemetry Data Platform ingère plusieurs types de signaux dans un *data lake* unifié. La fonctionnalité *New Relic AI* (NR AI) propose une détection d'anomalies basée sur des algorithmes de machine learning propriétaires.

Limites : modèle de facturation à la consommation de données imprévisible pour les charges variables; l'IA reste un moteur de classification, sans explication en langage naturel.

3.1.4 Dynatrace

Dynatrace se distingue par son moteur *Davis AI*, un système d'analyse causale automatique qui corrèle les anomalies à travers la pile applicative complète. Davis est capable de remonter automatiquement la chaîne de causalité d'un incident depuis le symptôme jusqu'à la cause racine.

Points forts : analyse causale la plus avancée du marché, autodécouverte des topologies.

Limites : tarification extrêmement élevée (de l'ordre de 69 \$/mois par *full-stack host*); propriétaire et fermé; ne convient pas aux petites structures.

3.1.5 Splunk + ITSI

Splunk Enterprise est une plateforme d'analyse de journaux et de métriques massivement utilisée en environnement d'entreprise. Son module *IT Service Intelligence* (ITSI) [9] ajoute des fonctionnalités de détection d'anomalies basées sur le *Machine Learning Toolkit* (MLTK) et des corrélations d'événements. CAIMP propose une intégration optionnelle avec Splunk comme source de métriques secondaire, permettant aux organisations déjà équipées de Splunk de bénéficier de l'IA explicative de CAIMP.

3.1.6 Alternatives open source : SigNoz, Netdata, Zabbix, Nagios

SigNoz Alternative open source à Datadog construite sur ClickHouse, supportant métriques, traces et journaux. Bonne interface, mais pas d'IA explicative.

Netdata Monitoring temps réel à très faible latence, orienté serveur individuel, sans analyse multi-tenant.

Zabbix Plateforme mature (depuis 2001) aux capacités étendues mais à la courbe d'apprentissage très abrupte; pas d'IA native.

Nagios Pionnier du monitoring (1999), aujourd'hui techniquement vieilli ; approche purement par seuils statiques.

3.2 Tableau comparatif

Table 3.1 – Comparaison des solutions de supervision selon 12 critères

Critère	Datadog	Grafana/Pydmatrace	Edynatrace	SigNoz	Zabbix	caimp
Prix (mois/hôte)	15–31 \$	Gratuit	69 \$	Gratuit	Gratuit	Gratuit
On-premise	Non	Oui	Non	Oui	Oui	Oui
IA native	Partielle	Non	Avancée	Non	Non	Oui
LLM explicatif	Non	Non	Non	Non	Non	Oui
RAG + runbooks	Non	Non	Non	Non	Non	Oui
Corrélation chang.	Non	Non	Partielle	Non	Non	Oui
Prévision 24h	Oui	Non	Oui	Non	Non	Oui
Dédup. alertes	Oui	Via AM	Oui	Partielle	Non	Oui
Multi-tenant	Oui	Limite	Oui	Oui	Oui	Oui
Déploiement <10 min	Non	Non	Non	Non	Non	Oui
Dashboard answers-first	Non	Non	Partielle	Non	Non	Oui
Tableau de bord	Excellent	Excellent	Bon	Correct	Vieilli	Moderne

3.3 Techniques de détection d'anomalies

3.3.1 Détection par seuil statique

La détection par seuil est la méthode la plus simple : une alerte est déclenchée quand une valeur dépasse un seuil prédéfini. Sa simplicité est un atout, mais elle présente deux défauts majeurs : elle génère des faux positifs pour les systèmes à charge variable et ne s'adapte pas au comportement normal évolué d'un système.

CAIMP implémente cette méthode comme premier filtre rapide, avec des seuils configurables par variable d'environnement.

3.3.2 Score Z et détection statistique

Le Z-score mesure l'écart d'une observation à la moyenne en unités d'écart-type :

$$z = \frac{x - \mu}{\sigma} \quad (3.1)$$

où μ est la moyenne empirique calculée sur une fenêtre historique et σ l'écart-type. Une valeur $|z| > 3$ est considérée comme anormale dans la distribution normale standard (probabilité de 0,27 %). Cette méthode s'adapte automatiquement au comportement normal du système.

3.3.3 Détection par taux de variation

Cette méthode calcule la dérivée discrète de la série temporelle :

$$\Delta_t = \frac{x_t - x_{t-1}}{x_{t-1}} \times 100\% \quad (3.2)$$

Elle est particulièrement efficace pour détecter les saturations rapides (montée soudaine du CPU lors d'un *fork bomb*) que le Z-score pourrait manquer sur une fenêtre historique longue.

3.3.4 Forêt d'isolation (Isolation Forest)

L'Isolation Forest [6] est un algorithme non supervisé basé sur la construction d'arbres de décision aléatoires. Son principe : les observations aberrantes sont plus facilement *isolées* par des partitions aléatoires. Le score d'anomalie est inversement proportionnel à la profondeur moyenne d'isolation. Bien que non implémenté dans CAIMP v2, cet algorithme est pressenti pour les versions futures.

3.3.5 Lissage exponentiel de Holt-Winters

La méthode de Holt-Winters [7, 10] (ou double lissage exponentiel linéaire) modélise une série temporelle comme la somme d'un niveau et d'une tendance linéaire :

$$L_t = \alpha x_t + (1 - \alpha)(L_{t-1} + T_{t-1}) \quad (3.3)$$

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1} \quad (3.4)$$

$$\hat{x}_{t+h} = L_t + h \cdot T_t \quad (3.5)$$

où :

- L_t est le niveau lissé au temps t ;
- T_t est la tendance lissée au temps t ;
- $\alpha \in [0, 1]$ est le paramètre de lissage du niveau ;
- $\beta \in [0, 1]$ est le paramètre de lissage de la tendance ;
- $\hat{x}_{t+h}$ est la prévision à h pas dans le futur.

Dans CAIMP, les paramètres retenus sont $\alpha = 0.3$ et $\beta = 0.1$, favorisant un lissage modéré qui réagit aux tendances sans amplifier le bruit. Les intervalles de confiance sont calculés à $\pm 1.28\sigma$ des résidus (couverture approximative de 80 %).

3.4 Modèles de langage pour l'AIOPS

3.4.1 Panorama des LLM ouverts

L'émergence des modèles de langage à grande échelle (LLM) ouverts a ouvert une nouvelle dimension dans l'AIOPS : l'explication en langage naturel des incidents. Parmi les modèles les plus pertinents pour une exécution locale :

Llama 3.1 8B Le modèle retenu par CAIMP. Publié par Meta AI en 2024 [11], il offre un

excellent rapport qualité/performance en version quantifiée Q4_K_M : 5 Go de VRAM, latence inférieure à 2 secondes par requête sur du matériel courant.

Mistral 7B Comparable à Llama 3 en termes de performance, avec une fenêtre de contexte étendue de 32K tokens.

Phi-3 Mini (3.8B) Modèle compact de Microsoft, excellent pour l'inférence sur des machines à ressources limitées.

Gemma 7B Modèle de Google, performant mais moins bien aligné pour les tâches opérationnelles.

3.4.2 Quantification des modèles

La quantification consiste à réduire la précision numérique des poids d'un modèle (de float32 à int4, par exemple) pour diminuer son empreinte mémoire tout en conservant une qualité d'inférence acceptable [12]. Le format GGUF utilisé par Ollama permet la quantification Q4_K_M (4 bits, méthode K-means) qui réduit la taille de Llama 3.1 8B de 16 Go (float16) à environ 4.7 Go, rendant possible l'exécution sur un serveur standard.

3.4.3 Le pattern RAG (Retrieval-Augmented Generation)

Le RAG [13] est une technique d'augmentation du contexte d'un LLM par la récupération dynamique de documents pertinents depuis une base de connaissances. Le schéma de fonctionnement est le suivant :

1. L'**indexation** : les documents (runbooks, incidents passés) sont convertis en vecteurs d'embedding via un modèle dédié (`nomic-embed-text`) et stockés dans une base vectorielle (pgvector [14]).
2. La **recherche** : lors d'un incident, un vecteur est calculé pour le contexte de l'anomalie, et les k documents les plus similaires sont récupérés par recherche de voisins approximatifs (cosine similarity).
3. L'**augmentation** : les documents récupérés sont injectés dans le prompt du LLM pour enrichir son contexte avant génération.

Ce pattern permet à un LLM à connaissance générale de s'adapter spécifiquement au contexte d'une organisation, sans *fine-tuning* coûteux.

3.5 Positionnement de CAIMP

L'analyse de l'état de l'art fait apparaître un espace vacant entre les solutions commerciales onéreuses (Datadog, Dynatrace) et les outils open source sans IA (Grafana/Prometheus). Cet espace est celui des *petites équipes* cherchant une plateforme auto-hébergée, explicative et accessible.

CAIMP se positionne sur cet espace avec les différenciants suivants :

- **Explication en langage naturel** via un LLM local (aucun outil open source ne propose cela) ;

- **Corrélation automatique des changements** (absente de toutes les solutions open source comparées) ;
- **Architecture événementielle moderne** avec NATS JetStream (plus légère que Kafka pour ce cas d'usage) ;
- **Déploiement en 10 minutes** sur n'importe quelle machine.

Conclusion

L'état de l'art révèle que si les outils de monitoring ne manquent pas, aucun ne combine simultanément l'auto-hébergement, l'accessibilité pour les non-experts et l'explication en langage naturel par LLM local. Cette lacune justifie pleinement la création de CAIMP. Le chapitre suivant traduit ces constats en spécifications fonctionnelles et non fonctionnelles concrètes.

Chapitre 4

Analyse et spécification des besoins

Introduction

Ce chapitre formalise les exigences du système CAIMP en besoins fonctionnels et non fonctionnels, identifie les acteurs, et présente les modèles UML qui guident la phase de conception. La démarche d'analyse s'est appuyée sur l'étude comparative du chapitre précédent, complétée par des scénarios d'usage concrets.

4.1 Identification des acteurs

Le système CAIMP implique quatre catégories d'acteurs :

Administrateur

Rôle le plus privilégié. Gère les organisations, les utilisateurs, les serveurs, et configure la plateforme. Accède à l'API d'administration (port 8001).

Opérateur Utilisateur technique qui consulte les tableaux de bord, gère les runbooks, configure les règles d'alerte, et interprète les analyses AI. Rôle **devops** en base.

Viewer Lecture seule. Consulte les dashboards et les rapports PDF. Rôle conçu pour les parties prenantes non techniques (managers, clients).

Agent système

Acteur non humain. Le processus démon **caimp-agent** s'exécute sur les serveurs surveillés. Il s'authentifie par certificat mTLS et envoie les métriques via OTLP.

4.2 Besoins fonctionnels

BF1 L'agent doit collecter les métriques CPU, mémoire et disque toutes les 15 secondes et les transmettre via OTLP/HTTP.

BF2 L'agent doit s'auto-inscrire auprès de la plateforme à l'aide d'un jeton d'enrôlement à usage unique.

- BF3** L'agent doit utiliser un certificat mTLS délivré par l'autorité de certification interne pour authentifier chaque connexion.
- BF4** L'agent doit détecter et reporter automatiquement les événements Docker (démarrage, arrêt, tirage d'image).
- BF5** L'agent doit surveiller les journaux d'installation de paquets (dpkg, yum) et les événements SSH depuis les journaux d'authentification.
- BF6** La plateforme doit détecter les anomalies en temps réel à l'aide d'au moins trois algorithmes distincts (seuil, Z-score, taux de variation).
- BF7** Les anomalies détectées doivent être publiées sur NATS JetStream pour traitement asynchrone par les services consommateurs.
- BF8** Le moteur d'analyse IA doit interroger la base vectorielle pour retrouver des incidents similaires passés et les runbooks pertinents.
- BF9** Le moteur d'analyse IA doit soumettre un prompt structuré à Ollama et valider la réponse JSON avant persistance.
- BF10** L'explication IA doit contenir : l'explication en langage naturel, la cause racine probable, l'action recommandée, et le niveau de confiance.
- BF11** Le moteur de corrélation doit interroger la table `change_events` dans la fenêtre de 30 minutes précédant chaque anomalie.
- BF12** Le moteur de corrélation doit scorer chaque changement par proximité temporelle et type, et produire un classement des causes probables.
- BF13** Le moteur de prévision doit calculer les prévisions Holt-Winters sur 24 heures pour chaque combinaison serveur/métrique.
- BF14** Le moteur de prévision doit calculer le *time to critical* — le délai prévu avant dépassement de seuil à 90 %.
- BF15** La déduplication d'alertes doit supprimer les alertes identiques (même serveur, même métrique, même détecteur) répétées dans les 15 minutes.
- BF16** L'API de requête doit exposer un endpoint `/dashboard/summary` agrégeant le statut global, les problèmes potentiels et les actions recommandées.
- BF17** L'interface utilisateur doit présenter en première page le statut global, les problèmes potentiels triés par criticité, et les actions recommandées numérotées.
- BF18** La plateforme doit permettre la gestion des runbooks (créer, modifier, supprimer) avec ré-indexation automatique des embeddings.
- BF19** Le module de rapports doit générer un rapport PDF multi-sections lisible par des non-techniciens, avec logo de l'organisation.

- BF20** L'interface doit proposer un chatbot SSE permettant de poser des questions en langage naturel sur les incidents et les serveurs.
- BF21** La plateforme doit recevoir les webhooks GitHub et GitLab pour les événements de déploiement.
- BF22** L'administrateur doit pouvoir configurer les canaux de notification (Slack, PagerDuty) pour les alertes critiques.
- BF23** Les mises à jour d'état doivent être poussées en temps réel vers les navigateurs connectés via WebSocket.
- BF24** La plateforme doit supporter l'isolation multi-tenant : les données d'une organisation ne doivent jamais être accessibles par une autre.
- BF25** La plateforme doit fournir un endpoint de santé (`/health`) pour chaque service, utilisé par les healthchecks Docker.

4.3 Besoins non fonctionnels

4.3.1 Performance

- **Latence d'ingestion** : Le Telemetry Writer doit traiter un batch de métriques en moins de 50 ms en conditions normales.
- **Throughput** : La plateforme doit supporter au moins 1 000 points de métriques par seconde sur un déploiement standard.
- **Temps de réponse API** : Les endpoints de lecture (GET) doivent répondre en moins de 300 ms pour un volume de 50 serveurs.
- **Latence IA** : Une explication complète doit être générée en moins de 30 secondes (dépendant du modèle LLM et du matériel).

4.3.2 Sécurité

- Authentification de chaque agent par certificat mTLS (TLS 1.3) ;
- Tokens d'enrôlement à usage unique, expirés après 30 minutes ;
- Jetons JWT HS256 avec expiration courte (15 minutes) pour les API ;
- Isolation des données par organisation via RLS PostgreSQL ;
- Chiffrement AES-256-GCM des secrets Splunk en base de données ;
- Journalisation immuable de toutes les actions administratives (`audit_log`).

4.3.3 Disponibilité et résilience

- Objectif de disponibilité : 99,5 % (temps d'arrêt maximum de 3h38/mois) ;
- Healthchecks Docker sur chaque service avec politique de redémarrage `unless-stopped` ;
- Buffering local de l'agent sur SQLite en cas d'indisponibilité temporaire de la plateforme ;

- Messages NATS JetStream persistés sur disque (durabilité garantie).

4.3.4 Maintenabilité

- Chaque service est autonome, déployable et testable indépendamment ;
- Migrations de base de données versionnées avec `golang-migrate` ;
- Conventions de commit standardisées (*Conventional Commits*) ;
- Documentation inline pour les fonctions publiques.

4.4 Diagramme de cas d'utilisation global

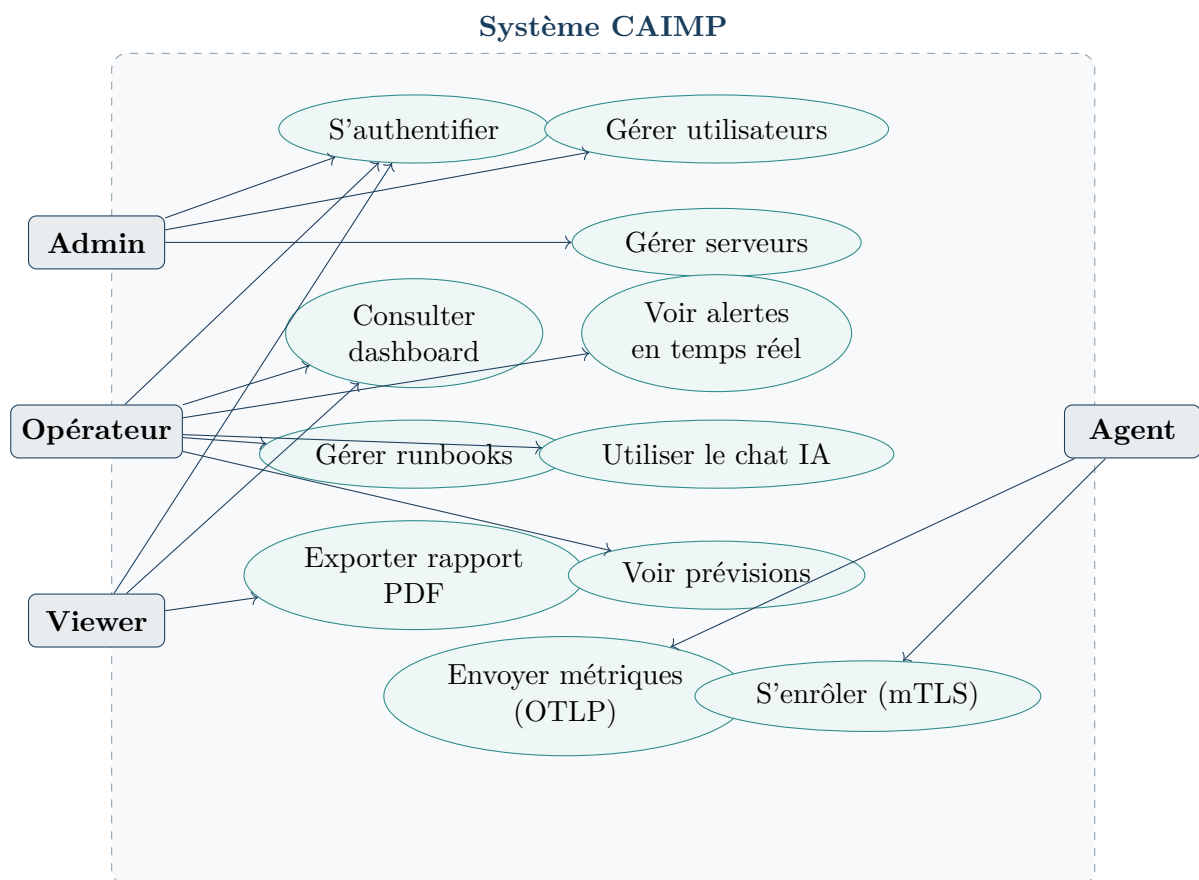


Figure 4.1 – Diagramme de cas d'utilisation global de CAIMP

4.5 Diagrammes de séquence

4.5.1 Enrôlement d'un agent

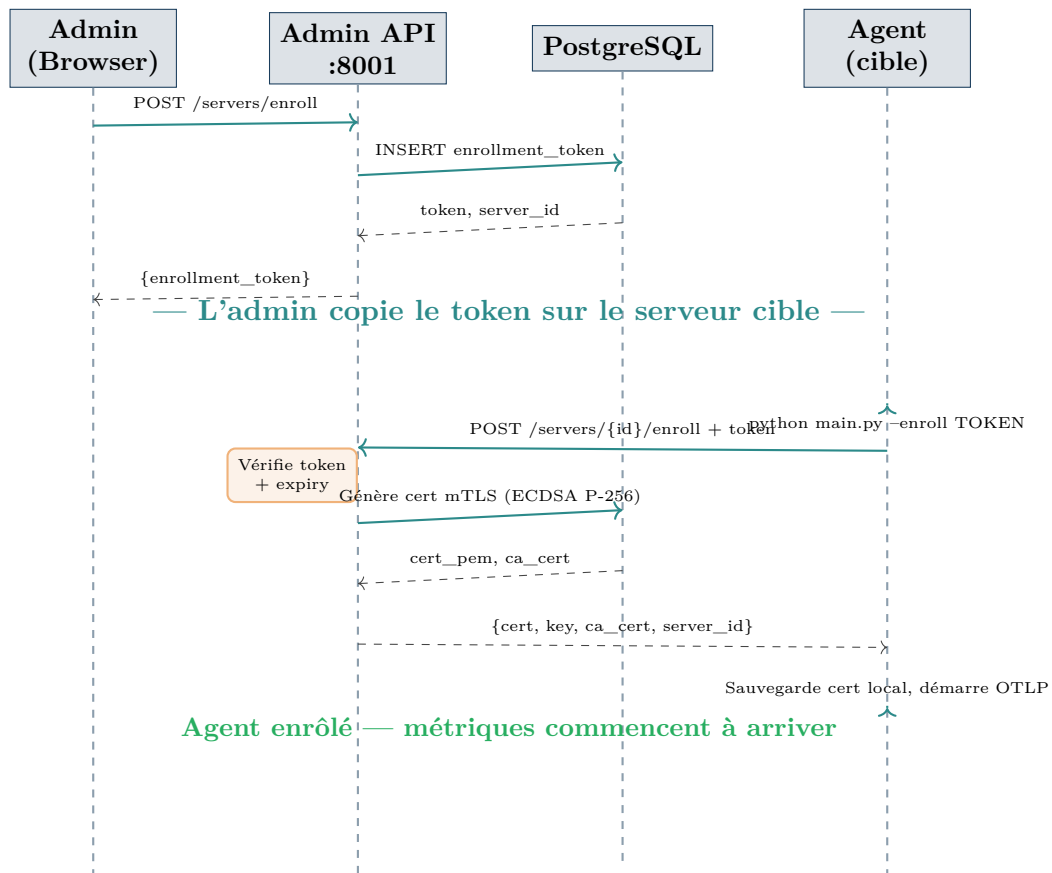


Figure 4.2 – Séquence d'enrôlement d'un agent de supervision

4.5.2 Détection d'anomalie et explication IA

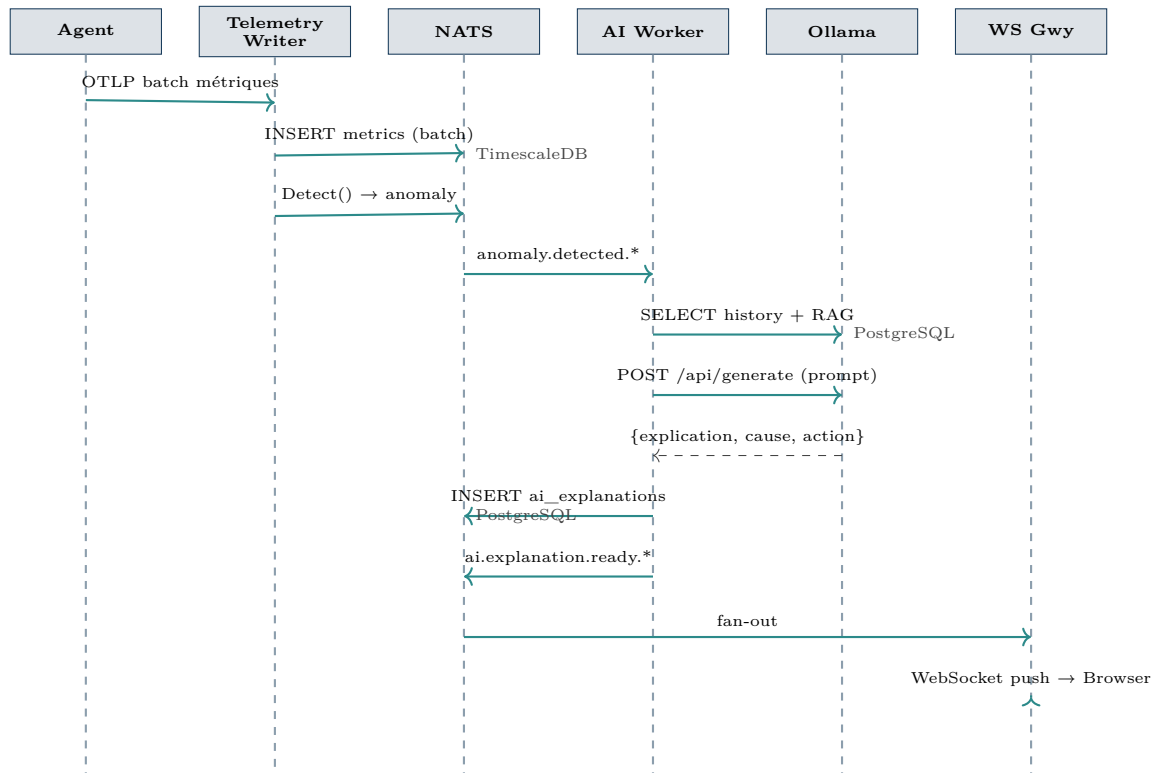


Figure 4.3 – Séquence de détection d'anomalie et génération d'explication IA

4.6 Matrice de traçabilité des besoins

Table 4.1 – Extrait de la matrice de traçabilité besoins fonctionnels – composants

BF	Besoin	Composant	Module
BF1	Collecte métriques OTLP	caimp-agent	collectors.py
BF2	Enrôlement par jeton	admin-api	enrollment.py
BF3	mTLS agent	otelcol / tw	Config mTLS
BF6	Détection anomalies (3 algos)	telemetry-writer	detector.go
BF7	Publication NATS	telemetry-writer	publisher.go
BF8–BF10	Explication IA	ai-worker	prompt_builder.py
BF11–BF12	Corrélation changements	correlation-engine	main.py
BF13–BF14	Prévision Holt-Winters	forecast-engine	forecaster.py
BF15	Déduplication alertes	alert-engine	worker.py
BF16–BF17	Dashboard answers-first	query-api	dashboard.py
BF18	Gestion runbooks + RAG	query-api	incidents.py
BF19	Rapport PDF	query-api	reports.py
BF21	Webhooks GitHub/GitLab	change-tracker	main.py
BF23	WebSocket push	ws-gateway	main.py
BF24	Isolation multi-tenant	PostgreSQL	Migrations RLS

Conclusion

L'analyse des besoins a permis d'identifier 25 besoins fonctionnels couvrant la totalité du périmètre de CAIMP, et de formaliser les exigences non fonctionnelles en matière de performance, sécurité, disponibilité et maintenabilité. Les diagrammes UML présentés constituent le contrat entre l'analyse et la conception qui fait l'objet du chapitre suivant.

Chapitre 5

Conception et architecture

Introduction

Ce chapitre est le cœur technique du rapport. Il décrit l'architecture microservices de CAIMP, le modèle de données, les flux de traitement principaux et les décisions de conception qui ont guidé le développement. Chaque choix architectural est justifié par rapport aux alternatives évaluées.

5.1 Choix architectural : microservices vs. monolithe

Face aux besoins de CAIMP, deux approches architecturales ont été évaluées : un monolithe modulaire et une architecture microservices. Le tableau 5.1 synthétise l'analyse comparative.

Table 5.1 – Comparaison monolithe vs. microservices pour CAIMP

Critère	Monolithe modulaire	Microservices
Déploiement	Simple (un processus)	Complexe (Docker Compose)
Scalabilité	Verticale uniquement	Horizontale par service
Isolation pannes	Un crash affecte tout	Dégradation gracieuse
Langages mixtes	Impossible	Possible (Go + Python)
Découplage IA	LLM dans le processus API	Worker dédié, indépendant
Évolutivité	Refactoring difficile	Ajout de services isolé

L'architecture microservices a été retenue pour trois raisons déterminantes : (1) le moteur de prévision Holt-Winters et l'inférence LLM sont des tâches de calcul intensif qui ne doivent pas bloquer les API de requête ; (2) le télétransmetteur Go bénéficie des goroutines pour l'ingestion concurrente sans les limitations du GIL Python ; (3) l'ajout futur de nouvelles sources de métriques (Kubernetes, AWS CloudWatch) ne nécessitera pas de modifier les services existants.

5.2 Architecture globale

La plateforme CAIMP est composée de 19 services conteneurisés organisés en quatre couches fonctionnelles :

Couche d'ingestion Reçoit les métriques des agents (`telemetry-writer`, `splunk-hec-bridge`) et les changements (`change-tracker`).

Couche d'événements Bus NATS JetStream assurant la communication asynchrone entre tous les services.

Couche de traitement Services consommateurs (`ai-worker`, `alert-engine`, `forecast-engine`, `correlation-engine`) traitant les événements de manière découplée.

Couche de présentation APIs REST (`query-api`, `admin-api`), WebSocket Gateway et frontend React.

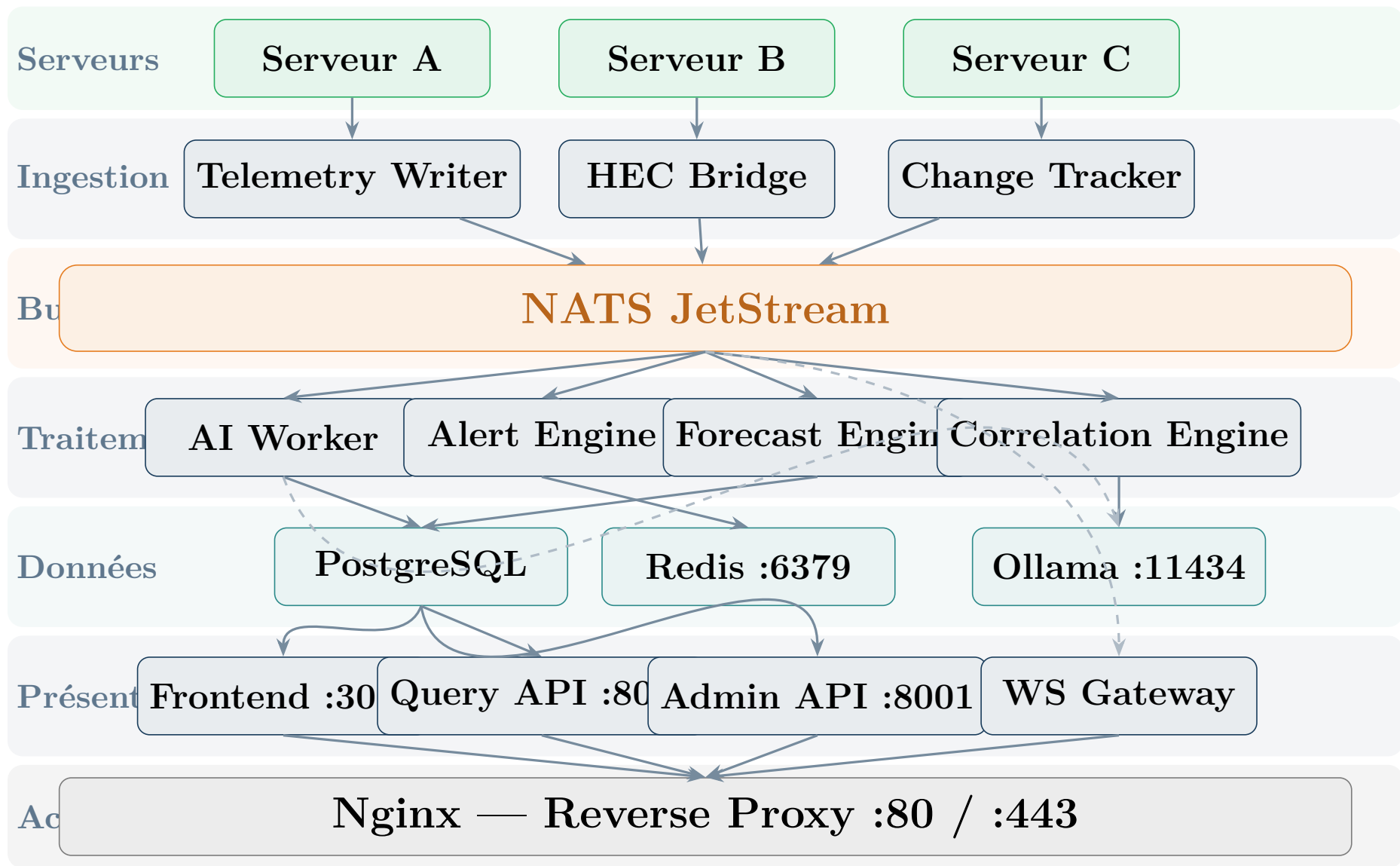


Figure 5.1 — Architecture globale de la plateforme CAIMP — 7 couches, 19 services

5.3 Fonctionnement de l'architecture — flux de bout en bout

La figure 5.1 présente la structure statique de la plateforme. Cette section décrit son fonctionnement dynamique : comment les données traversent chaque couche, depuis la collecte sur le serveur surveillé jusqu'à l'affichage d'une explication en langage naturel dans le navigateur de l'opérateur.

5.3.1 Flux 1 — Collecte et ingestion des métriques

L'agent CAIMP est un processus Python déployé sur chaque serveur cible. Toutes les 15 secondes, il lit les indicateurs système (CPU, mémoire, disque) via la bibliothèque `psutil` et les encode au format OTLP/HTTP (protobuf). Les données sont transmises au `Telemetry Writer` (port 4318), service Go qui :

1. **Reçoit** le batch OTLP et extrait les points de métrique ;
2. **Persiste** les valeurs dans la table `metrics` (hypertable TimescaleDB) via une insertion groupée (`COPY`) ;
3. **Détecte** les anomalies en parallèle grâce à trois algorithmes (seuil statique, Z-score, taux de variation) ;
4. **Publie** chaque anomalie détectée sur le sujet NATS `anomaly.detected.{org_id}.{severity}`.

En parallèle, l'agent exécute quatre *reporters* de changements qui surveillent en continu les événements Docker, les installations de paquets, les connexions SSH et les tâches cron. Ces événements sont envoyés au **Change Tracker** (port 8011), qui les inscrit dans la table `change_events` et les publie sur le sujet `change.detected.>`.

5.3.2 Flux 2 — Analyse IA et explication de l'anomalie

Dès qu'un message `anomaly.detected.*` arrive sur le bus NATS, **deux services consommateurs** s'activent de manière indépendante :

AI Worker Ce service abonne la file `ANOMALIES` et, pour chaque anomalie :

1. Interroge PostgreSQL pour récupérer les 30 dernières minutes de métriques (contexte historique du serveur) ;
2. Interroge pgvector par similarité cosinus pour retrouver les runbooks et les incidents passés les plus similaires (*Retrieval-Augmented Generation, RAG*) ;
3. Construit un prompt structuré ciblant un développeur non DevOps, en langage naturel, et l'envoie à Ollama (LLM Llama 3.1 8B) ;
4. Valide le JSON retourné par le modèle, l'insère dans `ai_explanations`, puis publie un message `ai.explanation.ready.*` sur NATS.

Correlation Engine Ce service effectue une analyse causale en parallèle :

1. Interroge `change_events` dans la fenêtre $[t - 30\text{min}, t]$ pour le même serveur ;

2. Score chaque changement candidat par la formule $s = e^{-0.1 \cdot \Delta t} \times w_{\text{type}}$ (décroissance temporelle $\times$ poids du type de changement) ;
3. Envoie les 3 premiers candidats à Ollama avec le contexte de l'anomalie pour obtenir un narratif causal en langage naturel ;
4. Persiste le résultat dans `root_cause_analyses` et publie un message `rootcause.ready.*` sur NATS.

5.3.3 Flux 3 — Prédiction des ressources

Le **Forecast Engine** fonctionne de manière entièrement autonome, indépendamment des anomalies. Il exécute deux boucles périodiques :

Boucle de prédiction (toutes les 30 min) Pour chaque combinaison serveur/métrique, récupère les 6 dernières heures de données, applique le lissage exponentiel double de Holt-Winters, génère 24 points de prédiction horaire avec intervalles de confiance à $\pm 1.28\sigma$, calcule le *time-to-critical* (heure prévue de franchissement du seuil de 90 %), demande un narratif à Ollama, et persiste le tout dans `server_forecasts`.

Boucle d'évaluation (toutes les 60 min) Compare les prévisions passées aux valeurs réelles, calcule l'erreur absolue moyenne (MAE), et enrichit le prochain prompt LLM de la précision historique du modèle pour calibrer le niveau de confiance exprimé.

5.3.4 Flux 4 — Mise à jour en temps réel du navigateur

Le **WS Gateway** maintient des connexions WebSocket persistantes avec tous les navigateurs connectés. Il souscrit aux sujets NATS suivants :

- `anomaly.detected.>` — affiche l'alerte dans le fil d'événements ;
- `ai.explanation.ready.>` — met à jour la carte d'explication IA ;
- `rootcause.ready.>` — met à jour l'analyse causale ;
- `forecast.critical.>` — déclenche une notification urgente.

Dès qu'un message arrive sur l'un de ces sujets, le gateway le distribue en *fan-out* à tous les clients WebSocket de l'organisation concernée (isolation multi-tenant respectée). Le tableau de bord se rafraîchit ainsi sans aucun *polling* — la latence entre détection et affichage est inférieure à 3 secondes.

5.3.5 Flux 5 — Requête utilisateur depuis le tableau de bord

Lorsqu'un opérateur ouvre le tableau de bord, le navigateur envoie une requête `GET /api/v1/dashboard/summary` au Query API (via Nginx). Celui-ci :

1. Récupère les statuts serveurs et les dernières métriques (cache Redis 30 s) ;
2. Agrège les anomalies des dernières 24 heures ;

3. Consulte les prévisions à risque ($time_to_critical \leq 24$ h) ;
4. Récupère les dernières analyses de cause racine depuis `root_cause_analyses` ;
5. Construit et retourne la réponse structurée : `overall_status`, `potential_issues` et `recommended_actions`.

L'interface présente directement le résultat sous forme de « tableau de bord orienté réponses » : statut global, problèmes potentiels triés par criticité, et actions recommandées numérotées.

5.4 Bus d'événements : NATS JetStream

5.4.1 Choix du bus de messages

Trois solutions de messagerie ont été évaluées pour le bus d'événements de CAIMP : Apache Kafka, RabbitMQ, et NATS JetStream.

Table 5.2 – Comparaison des solutions de messagerie

Critère	Kafka	RabbitMQ	NATS JetStream
Empreinte mémoire	~512 Mo (+ ZooKeeper)	~128 Mo	~30 Mo
Latence p99	2–10 ms	1–5 ms	< 1 ms
Persistance	Oui (disque)	Optionnelle	Oui (disque)
Multi-consumer	Pull (Consumer Groups)	Push	Pull et Push
Configuration	Complexe	Modérée	Simple
Protocole	Binaire propriétaire	AMQP	NATS natif

NATS JetStream [15] a été retenu pour sa faible empreinte mémoire (critique sur des machines de 4–8 Go de RAM), sa latence sub-milliseconde, et sa simplicité de configuration tout en offrant la durabilité nécessaire (messages persistés sur disque, accusés de réception).

5.4.2 Streams et sujets

La plateforme définit 8 streams JetStream, chacun couvrant un domaine fonctionnel :

Table 5.3 – Streams NATS JetStream de CAIMP

Stream	Sujets	Description	Rétention	Cons.
METRICS	<code>metric.received.></code>	Batches de métriques brutes	24h	2
ANOMALIES	<code>anomaly.detected.></code>	Anomalies détectées	7j	3
AI_OUT	<code>ai.explanation.ready.></code>	Explications IA générées	30j	1
CHAT	<code>chat.requested.></code>	Requêtes de chat	1h	1
AUDIT	<code>audit.log.></code>	Journal d'audit	90j	0
FORECASTS	<code>forecast.></code>	Prévisions générées	30j	1
CHANGES	<code>change.detected.></code>	Changements système	7j	2
ROOT_CAUSE	<code>rootcause.ready.></code>	Analyses de cause racine	30j	1

5.5 Modèle de données

5.5.1 Principes de conception

Le modèle de données de CAIMP repose sur PostgreSQL 16 avec les extensions TimescaleDB (données temporelles) et pgvector (recherche vectorielle). Deux principes fondamentaux ont guidé sa conception :

1. **Isolation multi-tenant par RLS** : chaque table portant des données applicatives contient une colonne `org_id`. Les politiques *Row-Level Security* garantissent qu'une requête ne peut accéder qu'aux lignes de l'organisation courante (définie par la fonction `current_org_id()`).
2. **Hypertables pour les données temporelles** : les tables `metrics` et `change_events` sont des hypertables TimescaleDB partitionnées automatiquement par tranches de temps. TimescaleDB applique une compression columnar automatique après 7 jours, réduisant l'espace de stockage d'un facteur 10 à 20.

5.5.2 Schéma relationnel

Le modèle complet comporte 17 tables, présentées par domaine fonctionnel :

Domaine authentification et gestion

- `organizations` : organisation multi-tenant (identifiant, nom, plan)
- `users` : comptes utilisateurs avec hash bcrypt, rôle RBAC, 2FA optionnel
- `servers` : serveurs enregistrés (hostname, statut, IP, labels JSONB)
- `ca_config` : configuration de l'autorité de certification interne
- `agent_certs` : certificats mTLS émis par la CA
- `enrollment_tokens` : jetons d'enrôlement à usage unique
- `refresh_token_blacklist` : jetons JWT révoqués

- `audit_log` : journal immuable de toutes les actions administratives

Domaine métriques et anomalies

- `metrics` : hypertable TimescaleDB — chaque ligne est un point de métrique (`server_id`, `metric_name`, `value`, `time`)
- `anomaly_events` : anomalies détectées avec détecteur, sévérité, valeur, seuil dépassé
- `alert_rules` : règles d'alerte configurables (seuil, cooldown)
- `notifications` : historique des notifications envoyées

Domaine intelligence artificielle

- `ai_explanations` : explications LLM (explication, cause, action, confiance, modèle utilisé)
- `server_forecasts` : prévisions Holt-Winters (points JSON, pente, time-to-critical, narratif LLM)
- `rag_documents` : documents indexés pour RAG avec vecteur `VECTOR(768)` et index HNSW cosinus
- `runbooks` : guides de remédiation avec contenu et vecteur d'embedding

Domaine corrélation de changements

- `change_events` : hypertable — tous les changements système (type, source, acteur, charge utile JSONB, SHA git, SHA image)
- `root_cause_analyses` : sorties du moteur de corrélation (cause probable, score de confiance, changements corrélés JSONB, narratif)

Domaine Splunk

- `splunk_incidents` : incidents reçus via webhook Splunk
- `splunk_ai_explanations` : pipeline IA pour les incidents Splunk
- `ai_feedback` : notations thumbs-up/down des opérateurs
- `org_config` : configuration par organisation (LLM cloud, credentials Splunk)

5.5.3 Diagramme entité-association

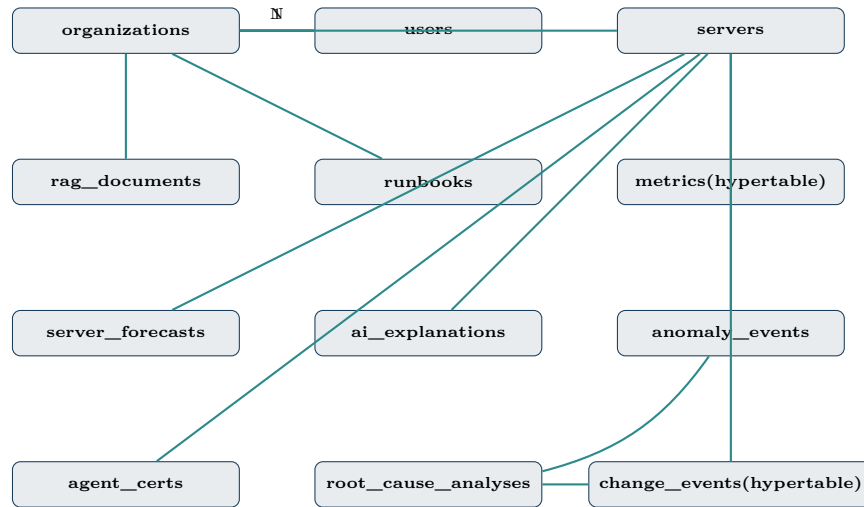


Figure 5.2 – Diagramme entité-association simplifié de CAIMP (17 tables)

5.6 Flux de traitement détaillés

5.6.1 Flux 1 : Ingestion de métriques

L’agent collecte les métriques toutes les 15 secondes et les envoie au Telemetry Writer via OTLP/HTTP. Le Telemetry Writer exécute les opérations suivantes dans une goroutine Go :

1. Décodage du batch OTLP et normalisation des noms de métriques (`system.cpu.utilization` → stocké tel quel) ;
2. Insertion en bloc dans la table `metrics` via COPY PostgreSQL (batch de 2 000 lignes maximum) ;
3. Passage des points au détecteur d’anomalies (trois algorithmes en parallèle) ;
4. Publication de chaque anomalie détectée sur le sujet `anomaly.detected.{org_id}.{severity}`.

5.6.2 Flux 2 : Corrélation de changements

1. Le Correlation Engine reçoit un message `anomaly.detected.*` ;
2. Il exécute une requête SQL sur `change_events` dans la fenêtre $[t - 30\text{min}, t]$ pour le même serveur ;
3. Chaque changement est scoré : $s = e^{-0.1 \cdot \Delta t} \times w_{\text{type}}$, où Δt est l’écart en minutes et $w_{\text{type}} \in [0.3, 1.0]$ selon le type de changement ;
4. Les trois changements de score le plus élevé sont envoyés à Ollama avec le contexte de l’anomalie ;
5. Le résultat est persisté dans `root_cause_analyses` et publié sur `rootcause.ready.{org_id}`.

5.7 Sécurité : modèle de menaces STRIDE

L'analyse STRIDE (*Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege*) identifie les menaces suivantes et les contre-mesures appliquées dans CAIMP :

Table 5.4 – Analyse STRIDE et contre-mesures de CAIMP

Menace	Exemple concret	Contre-mesure
Spoofing	Agent malveillant envoyant des métriques falsifiées	Certificat mTLS ECDSA P-256 ; jeton JWT court
Tampering	Modification des métriques en transit	TLS 1.3 ; intégrité AEAD
Repudiation	Nier une action administrative	Journal d'audit immuable (audit_log)
Info Disclosure	Accès aux données d'une autre organisation	RLS PostgreSQL ; isolation réseau Docker
DoS	Flood de métriques depuis un agent compromis	Rate limiting Nginx (200 req/-min/IP)
EoP	Utilisateur <code>viewer</code> accédant aux routes admin	RBAC : vérification du rôle dans chaque endpoint

Conclusion

Ce chapitre a présenté les fondations architecturales de CAIMP : le choix de l'architecture microservices, l'organisation des 19 services en quatre couches, le bus d'événements NATS JetStream, le modèle de données relationnel et les principaux flux de traitement. Cette architecture assure la séparation des préoccupations nécessaire à l'évolutivité et à la maintenabilité de la plateforme. Le chapitre suivant justifie les choix technologiques qui donnent corps à cette architecture.

Chapitre 6

Étude des technologies utilisées

Introduction

Chaque technologie intégrée dans CAIMP a fait l'objet d'une évaluation systématique : pertinence fonctionnelle, maturité, communauté, empreinte mémoire, et cohérence avec l'ensemble du système. Ce chapitre présente et justifie chacun de ces choix.

6.1 Langages de programmation

6.1.1 Python 3.12

Python est le langage dominant pour les applications d'intelligence artificielle, notamment grâce à l'écosystème NumPy/SciPy et aux bibliothèques de traitement de texte. Dans CAIMP, il est utilisé pour les services API (FastAPI), les workers asynchrones (AI Worker, Alert Engine, Forecast Engine, Correlation Engine), et l'agent de supervision.

La version 3.12 apporte des gains de performance significatifs (10–15 % sur les benchmarks CPython) et améliore les messages d'erreur, facilitant le débogage. La syntaxe `asyncio` (`async/await`) est utilisée pervasively pour la programmation asynchrone, indispensable pour les services qui attendent simultanément des messages NATS et des réponses Ollama.

Alternatives évaluées : Node.js (moins adapté à NumPy), Ruby on Rails (absence d'écosystème IA), Java (verbosité excessive pour ce type de service).

6.1.2 Go 1.22

Go a été retenu pour le Telemetry Writer, le service le plus critique en termes de performance. Son modèle de concurrence basé sur les goroutines et les channels est idéal pour l'ingestion parallèle de métriques depuis de multiples agents. Un serveur Go démarre en quelques millisecondes et consomme moins de 20 Mo de RAM au repos, contre plusieurs centaines pour une application JVM équivalente.

Justification spécifique : le Telemetry Writer traite des batches OTLP en désérialisant des buffers protobuf, en écrivant en base via COPY PostgreSQL, et en publiant sur NATS — trois opérations qui se prêtent naturellement aux goroutines. Le compilateur Go

génère un binaire statique, simplifiant le packaging Docker.

6.1.3 TypeScript (React 18)

Le frontend est écrit en TypeScript, sur-ensemble typé de JavaScript, compilé par Vite pour la production. Le typage statique est particulièrement précieux pour un projet comme CAIMP où les interfaces entre frontend et backend sont nombreuses et évolutives.

6.2 Frameworks backend

6.2.1 FastAPI

FastAPI [16] est un *framework* web Python asynchrone basé sur Starlette et Pydantic. Ses avantages décisifs pour CAIMP :

- **Génération automatique de documentation OpenAPI/Swagger**, accessible à `/docs` pour chaque service ;
- **Validation automatique des payloads** via les modèles Pydantic, éliminant le code de validation manuel ;
- **Support natif de `async/await`**, compatible avec `asyncpg` et `nats-py` ;
- **Performances** comparables à Node.js (benchmarks TechEmpower).

Alternatives évaluées : Flask (synchrone par défaut, verbeux), Django REST Framework (trop lourd pour des microservices), Litestar (moins mature).

6.2.2 `asyncpg`

`asyncpg` est un pilote PostgreSQL `asyncio` natif écrit en C, offrant des performances nettement supérieures à `psycopg2` (2 à 3 fois plus rapide en lecture). Son support du protocole *binary* PostgreSQL élimine la sérialisation des données en texte, réduisant la latence des requêtes fréquentes.

6.3 Base de données

6.3.1 PostgreSQL 16 + TimescaleDB

PostgreSQL [17] est le système de gestion de bases de données relationnelles le plus avancé du monde *open source*. L'extension TimescaleDB [5] transforme PostgreSQL en base de données temporelles haute performance par trois mécanismes :

Hypertables

Partitionnement automatique par tranches de temps (chunks) sans modification de l'API SQL. Pour la table `metrics`, chaque chunk couvre 7 jours. Les requêtes sur un intervalle de temps ne lisent que les chunks pertinents (*chunk exclusion*).

Compression columnar

Après 7 jours, les chunks sont compressés en format columnar (similaire à

Parquet). Le taux de compression moyen pour les métriques numériques est de 12 :1, réduisant 100 Go de données brutes à moins de 9 Go.

Politiques de rétention

Des politiques automatiques suppriment les chunks anciens (par exemple, supprimer les métriques brutes de plus de 90 jours).

6.3.2 pgvector

pgvector [14] est une extension PostgreSQL permettant de stocker et d'interroger des vecteurs d'embedding dans des colonnes de type `VECTOR(n)`. CAIMP l'utilise pour deux types de documents :

- Les **runbooks** : guides de remédiation de l'équipe, indexés avec `nomic-embed-text` (768 dimensions) ;
- Les **rag_documents** : incidents passés avec leur résolution.

L'index HNSW (*Hierarchical Navigable Small World*) est utilisé pour la recherche approximative de voisins les plus proches par similarité cosinus. Sa complexité de recherche est $O(\log n)$ contre $O(n)$ pour la recherche exhaustive, permettant des requêtes sub-milliseconde sur des centaines de milliers de vecteurs.

6.3.3 Redis 7.2

Redis est utilisé comme cache distribué et pour les fonctionnalités temps réel :

- **Déduplication d'alertes** : clés NX avec TTL de 15 minutes ;
- **Cache API** : résultats de requêtes fréquentes (30 secondes) ;
- **Métriques live** : valeurs brutes des dernières 90 secondes pour l'affichage en temps réel (HEC Bridge) ;
- **Liste noire JWT** : jetons de rafraîchissement révoqués.

6.4 Intelligence artificielle

6.4.1 Ollama

Ollama est un serveur d'inférence LLM local exposant une API REST compatible avec l'interface OpenAI. Il supporte les formats GGUF et GGML, permettant l'exécution de modèles quantifiés sur du matériel standard. CAIMP l'utilise pour deux modèles :

- **llama3.1 :8b-instruct-q4_K_M** : modèle principal pour les explications d'anomalies et les narratifs de prévision. La quantification Q4_K_M (4 bits, méthode K-means) réduit la taille de 16 Go (fp16) à 4.7 Go avec une dégradation de qualité inférieure à 2 % sur les benchmarks de raisonnement [12] ;
- **nomic-embed-text** : modèle d'embedding de 270 Mo produisant des vecteurs de 768 dimensions, optimisé pour la recherche sémantique [18].

6.4.2 Holt-Winters — Formulation mathématique complète

Le lissage exponentiel double de Holt-Winters, implémenté en NumPy pur dans `forecaster.py`, suit les équations ci-dessous :

$$\begin{aligned}
 L_0 &= x_1 && \text{(initialisation du niveau)} \\
 T_0 &= \frac{x_n - x_1}{n - 1} && \text{(initialisation de la tendance)} \\
 L_t &= \alpha \cdot x_t + (1 - \alpha)(L_{t-1} + T_{t-1}), \quad 0 < \alpha < 1 && \text{(mise à jour du niveau)} \\
 T_t &= \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1}, \quad 0 < \beta < 1 && \text{(mise à jour de la tendance)} \\
 \hat{x}_{t+h} &= L_t + h \cdot T_t && \text{(prévision à h pas)}
 \end{aligned}$$

Les intervalles de confiance à $\pm 1.28\sigma$ (couvrant environ 80 % de la distribution) sont calculés sur les résidus $r_t = x_t - \hat{x}_{t|t-1}$:

$$\hat{x}_{t+h} \pm 1.28 \cdot \hat{\sigma}_r \quad (6.1)$$

où $\hat{\sigma}_r = \sqrt{\frac{1}{n} \sum_{t=1}^n r_t^2}$ est l'écart-type empirique des résidus. Le *time to critical* est calculé par interpolation linéaire sur la séquence des prévisions dès que $\hat{x}_{t+h} \geq 0.9$ (seuil de 90 %).

6.5 Infrastructure

6.5.1 Docker et Docker Compose v2

Docker [19] isole chaque service dans un conteneur immuable, garantissant la reproductibilité des déploiements. Docker Compose v2 orchestre les 19 services de CAIMP avec :

- **Healthchecks** sur chaque service, avec politique de dépendance (`condition: service_healthy`) pour garantir l'ordre de démarrage ;
- **Profils Docker** pour les composants optionnels (`-profile otel` pour l'OTel Collector, `-profile full` pour Ollama en mode test) ;
- **Volumes nommés** pour la persistance de PostgreSQL, Redis, NATS et Ollama ;
- **Réseau bridge isolé** (`caimp_net`) : aucun service n'est exposé directement sauf via Nginx.

6.5.2 Nginx

Nginx sert de reverse proxy centralisé, concentrant les responsabilités transversales :

- Terminaison TLS et redirections HTTPS $\rightarrow$ HTTP ;
- Rate limiting (200 req/min pour les API, 10 req/min pour l'authentification) ;
- En-têtes de sécurité (CSP, HSTS, X-Frame-Options) ;

— Proxy SSE sans buffering pour le chat IA (`proxy_buffering off`).

6.5.3 Protocole OTLP

L'OTLP [4] est le protocole standard de la CNCF pour la transmission de métriques, traces et journaux. CAIMP utilise OTLP/HTTP avec encoding protobuf entre l'agent et le Telemetry Writer. Le choix de ce standard ouvert garantit la compatibilité future avec d'autres *backends* d'observabilité et d'autres langages d'instrumentation.

6.6 Sécurité

6.6.1 JWT (JSON Web Token)

Les jetons JWT [20] sont utilisés pour l'authentification des requêtes API. CAIMP utilise l'algorithme HS256 (HMAC-SHA256) avec une clé secrète de 64 octets (512 bits), générée par `openssl rand -hex 64`. Les jetons d'accès expirent en 15 minutes, limitant la fenêtre d'exploitation en cas de vol. Les jetons de rafraîchissement (7 jours) sont révocables via la table `refresh_token_blacklist`.

6.6.2 mTLS pour les agents

L'authentification mutuelle TLS [21] garantit que seuls les agents possédant un certificat valide émis par la CA interne peuvent envoyer des métriques. L'autorité de certification est stockée chiffrée en base (AES-256-GCM) et n'est jamais exposée via les API. La révocation d'un agent s'effectue en temps réel via la table `revoked_agents`, consultée à chaque connexion.

Conclusion

Le choix de chaque technologie dans CAIMP résulte d'une évaluation rigoureuse des alternatives. L'association Python/Go pour le backend, TimescaleDB pour les données temporelles, NATS pour le bus d'événements, et Ollama pour l'inférence LLM locale constitue un ensemble cohérent, performant et accessible. Le chapitre suivant détaille l'implémentation concrète de ces choix.

Chapitre 7

Réalisation et implémentation

Introduction

Ce chapitre présente les détails d'implémentation des modules clés de CAIMP, illustrés par des extraits de code commentés. Nous décrivons les difficultés rencontrées, les solutions adoptées, et la répartition du travail au sein de l'équipe.

7.1 Organisation du dépôt

Le dépôt CAIMP v2 est organisé selon une structure monorepo, chaque service occupant son propre répertoire sous `services/` :

```
1 caimp/
2     services/
3         agent/ # Python : collecte + enr lement
4     mTLS
5         telemetry-writer/ # Go : ingestion OTLP + d tect ion
6         ai-worker/ # Python : NATS RAG Ollama
7         alert-engine/ # Python : r gles + d duplication
8         forecast-engine/ # Python : Holt-Winters + LLM
9         correlation-engine/ # Python : corr lation
10    changements
11        change-tracker/ # Python : webhooks + ingest
12        api-admin/ # Python FastAPI : auth + PKI
13        api-query/ # Python FastAPI : lecture + PDF
14        ws-gateway/ # Python : NATS WebSocket
15        ai-orchestrator/ # Python FastAPI : Splunk + Chat
16    infra/
17        postgres/migrations/ # 14 migrations SQL
18        nats/ # config + setup_streams.py
19        nginx/ # nginx.conf + certs
20    frontend/ # React 18 + Vite + TypeScript
21    libs/nats_client/ # Client NATS partag (Python)
22    scripts/ # seed_demo.sql, smoke_test
23    docker-compose.yml
24    Makefile
```

Listing 7.1 – Structure du dépôt CAIMP

7.2 Répartition des tâches

Table 7.1 – Répartition des responsabilités entre les membres de l'équipe

Membre	Modules principaux	Semaines
CHAHIDY Rim	Telemetry Writer (Go), Alert Engine, frontend React (Dashboard, Forecasts, Incidents), CSS/design system	S1–S4
SAMDI Wiam	AI Worker, AI Orchestrator, Correlation Engine, RAG pipeline, prompts LLM, PDF reports	S1–S4
TAÏK Youssef	Admin API, Query API, base de données (migrations RLS), Change Tracker, agent CAIMP, DevOps (Docker Compose, Makefile, scripts)	S1–S4
Travaux communs : architecture, revues de code, tests d'intégration, documentation		S1–S4

7.3 Module 1 : Agent de supervision (Python)

7.3.1 Collecte de métriques

L'agent utilise la bibliothèque `psutil` pour lire les métriques système de manière portable, puis les exporte via l'OpenTelemetry SDK :

```

1 from opentelemetry import metrics
2
3 def register_all(meter: metrics.Meter) -> None:
4     """Enregistre les collecteurs CPU, mmoire et disque."""
5
6     # CPU : utilisation en pourcentage (0.0-1.0)
7     cpu = meter.create_observable_gauge(
8         "cpu_percent",
9         callbacks=[lambda: [(psutil.cpu_percent(interval=1) / 100.0,
10 {})]],
11         description="CPU utilization",
12         unit="1",
13     )
14
15     # Mmoire virtuelle
16     mem = meter.create_observable_gauge(
17         "memory_percent",
18         callbacks=[lambda: [(psutil.virtual_memory().percent / 100.0,
19 {})]],
20         description="Memory utilization",
21         unit="1",
22     )

```

```
21
22     # Disque (partition principale)
23     def _disk():
24         try:
25             usage = psutil.disk_usage("/")
26             return [(usage.percent / 100.0, {})]
27         except PermissionError:
28             return [(0.0, {})]
29
30     disk = meter.create_observable_gauge(
31         "disk_percent",
32         callbacks=[_disk],
33         description="Disk utilization",
34         unit="1",
35     )
```

Listing 7.2 – Extrait de `caimp_agent/collectors.py`

7.3.2 Enrôlement mTLS

L'enrôlement est la séquence critique de sécurité. L'agent génère une paire de clés ECDSA P-256 localement, transmet la clé publique à l'Admin API, qui l'emballe dans un certificat signé par la CA interne.

7.4 Module 2 : Telemetry Writer (Go)

7.4.1 Détecteur d'anomalies

Le détecteur Go implémente les trois algorithmes de manière concurrente. Voici l'implémentation du détecteur Z-score :

```
1 // zScore calcule le score Z sur l'historique des 100 derni res
   valeurs.
2 func (d *Detector) zScore(
3     ctx context.Context,
4     points []receiver.MetricPoint,
5 ) []AnomalyEvent {
6     var events []AnomalyEvent
7     for _, p := range points {
8         // R cup re l'historique depuis PostgreSQL
9         rows, err := d.pool.Query(ctx, '
10             SELECT value FROM metrics
11             WHERE server_id=$1 AND metric_name=$2
12             ORDER BY time DESC LIMIT 100', p.ServerID, p.MetricName)
13         if err != nil {
14             continue
15         }
16         var vals []float64
17         for rows.Next() {
```

```
18     var v float64
19     if err := rows.Scan(&v); err == nil {
20         vals = append(vals, v)
21     }
22 }
23 rows.Close()
24 if len(vals) < 30 { // pas assez d'historique
25     continue
26 }
27 // Calcul et
28 mean, std := meanStd(vals)
29 if std < 1e-9 {
30     continue
31 }
32 z := math.Abs((p.Value - mean) / std)
33 if z >= d.cfg.ZScoreThreshold {
34     sev := "warning"
35     if z >= d.cfg.ZScoreThreshold*1.5 {
36         sev = "critical"
37     }
38     events = append(events, AnomalyEvent{
39         IncidentID: uuid.New().String(),
40         OrgID:       p.OrgID,
41         ServerID:    p.ServerID,
42         MetricName:  p.MetricName,
43         Value:       p.Value,
44         Detector:    "zscore",
45         Severity:    sev,
46         Timestamp:  p.Time,
47     })
48 }
49 }
50 return events
51 }
```

Listing 7.3 – Extrait du détecteur Z-score (detector.go)

7.5 Module 3 : AI Worker et pipeline RAG

7.5.1 Construction du prompt

Le prompt est construit en plusieurs étapes : récupération du contexte historique, recherche RAG dans pgvector, et assemblage.

```
1 def build_prompt(event: dict, metric_history: list[dict],
2                 rag_docs: list[dict]) -> str:
3     """
4     Construit un prompt ciblant un dveloppeur sans formation DevOps.
5     Les m triques sont affich es en pourcentages lisibles.
```

```

6      """
7      history_lines = "\n".join(
8          f"    {r['time']}: {r['value']:.1%}"
9          if r['value'] <= 1.0 else f"    {r['time']}: {r['value']:.1f}%"
10         for r in metric_history[:20]
11     ) or "    Pas d'historique disponible"
12
13     rag_lines = "\n\n".join(
14         f"[{d['type'].upper()}] {d['title'] or 'Incident pass '}: \n"
15         f"{d['content'][:400]}"
16         for d in rag_docs
17     ) or "    Aucun incident similaire en base"
18
19     pct = f"{event.get('value', 0) * 100:.1f}%"
20     return f"""Tu aides un dveloppeur sans formation DevOps...
21 ALERTE : {event.get('metric_name')} a atteint {pct}
22 HISTORIQUE : {history_lines}
23 INCIDENTS SIMILAIRES : {rag_lines}
24 {_OUTPUT_SCHEMA}"""

```

Listing 7.4 – Extrait de prompt_builder.py

7.6 Module 4 : Moteur de prévision (Holt-Winters)

7.6.1 Implémentation NumPy

L'implémentation du lissage exponentiel double est entièrement vectorisée pour minimiser la latence :

```

1  def _holt_linear(
2      values: np.ndarray, horizon: int,
3      alpha: float = 0.3, beta: float = 0.1
4  ) -> tuple[np.ndarray, float, np.ndarray]:
5      """Lissage exponentiel double. Retourne (prvisions, tendance,
6      ajust )."""
7      n = len(values)
8      # Initialisation
9      level = float(values[0])
10     trend = float((values[-1] - values[0]) / max(n - 1, 1))
11     fitted = np.empty(n)
12
13     for t in range(n):
14         fitted[t] = level + trend
15         if t < n - 1:
16             prev_level = level
17             level = alpha * float(values[t]) + (1.0 - alpha) * (level
18             + trend)
19             trend = beta * (level - prev_level) + (1.0 - beta) * trend

```

```

19     # Provisions : L_t + h * T_t
20     forecasts = np.array([level + (h + 1) * trend for h in range(
horizon)])
21     return forecasts, trend, fitted

```

Listing 7.5 — Extrait de forecaster.py — lissage de Holt-Winters

7.7 Module 5 : Moteur de corrélation de changements

La fonction de scoring implémente la formule $s = e^{-0.1\Delta t} \times w_{\text{type}}$:

```

1  # Poids par type de changement (plus = plus susceptible de causer une
   anomalie)
2  _TYPE_WEIGHT = {
3      "deployment": 1.0,
4      "config": 0.9,
5      "package": 0.8,
6      "docker_pull": 0.75,
7      "restart": 0.7,
8      "cron": 0.5,
9      "ssh_login": 0.3,
10 }
11
12 def _score_change(change: dict, anomaly_time: datetime) -> float:
13     """Calcule le score de corrélation d'un changement avec une
   anomalie."""
14     occurred_at = change["occurred_at"]
15     if occurred_at.tzinfo is None:
16         occurred_at = occurred_at.replace(tzinfo=timezone.utc)
17
18     # Délai en minutes entre le changement et l'anomalie
19     minutes_before = (anomaly_time - occurred_at).total_seconds() / 60
20     if minutes_before < 0:
21         return 0.0 # Changement postérieur à l'anomalie
22
23     # Croissance exponentielle : f(dt) = exp(-lambda * dt)
24     temporal = math.exp(-0.1 * max(0, minutes_before))
25     type_weight = _TYPE_WEIGHT.get(change["change_type"], 0.5)
26     return temporal * type_weight

```

Listing 7.6 — Fonction de scoring dans correlation-engine/main.py

7.8 Module 6 : Frontend React

7.8.1 Tableau de bord orienté réponses

Le composant central du nouveau tableau de bord est **StatusBanner**, qui affiche immédiatement le statut global sans nécessiter de lecture de graphes :

```

1 function StatusBanner({ summary }: { summary: DashboardSummary }) {
2   const isCritical = summary.overall_status === 'critical'
3   const isHealthy  = summary.overall_status === 'healthy'
4   const bg = isCritical ? 'var(--danger)' :
5             isHealthy  ? 'var(--success)' : 'var(--warning)'
6
7   return (
8     <div style={{ background: faint, border: '1.5px solid ${bg}',
9                 borderRadius: 14, padding: '20px 28px',
10                display: 'flex', alignItems: 'center', gap: 16 }}>
11       {/* Ic ne de statut */}
12       <div style={{ width: 52, height: 52, borderRadius: '50%',
13                 background: bg, display: 'flex',
14                 alignItems: 'center', justifyContent: 'center' }}>
15         {isCritical ? <AlertTriangle size={28} color="#fff" />
16           : <Check size={28} color="#fff" />}
17       </div>
18       {/* Message principal */}
19       <div>
20         <div style={{ fontSize: 18, fontWeight: 800, color: bg }}>
21           {isHealthy ? 'Tous les syst mes fonctionnent' :
22             isCritical ? 'Action requise' :
23               'Probl mes potentiels d tect s'}
24         </div>
25         <div style={{ fontSize: 13.5, color: 'var(--text-2)' }}>
26           {summary.overall_message}
27         </div>
28       </div>
29     </div>
30   )
31 }

```

Listing 7.7 – Extrait de Dashboard.tsx — composant StatusBanner

7.9 Module 7 : Génération de rapports PDF

Le rapport PDF est généré avec ReportLab Platypus. La page de couverture est dessinée directement sur le canvas de la première page :

```

1 def _cover_chrome(canvas, doc):
2   canvas.saveState()
3   # Fond brun fonc
4   canvas.setFillColor(colors.HexColor("#3d2b1f"))
5   canvas.rect(0, 0, PW, PH, fill=1, stroke=0)
6
7   # Cercles d coratifs
8   canvas.setFillColor(colors.HexColor("#5c4033"))
9   canvas.circle(PW + 8*mm, PH - 18*mm, 88*mm, fill=1, stroke=0)

```

```

10
11     # Logo "C" en badge arrondi
12     canvas.setFillColor(WHITE)
13     canvas.roundRect(MARGIN, PH*0.60, 22*mm, 22*mm, 3.5*mm, fill=1,
14 stroke=0)
15     canvas.setFillColor(colors.HexColor("#3d2b1f"))
16     canvas.setFont("Times-Bold", 17)
17     canvas.drawCentredString(MARGIN + 11*mm, PH*0.60 + 6.5*mm, "C")
18
19     # Titre du rapport
20     canvas.setFont("Helvetica-Bold", 34)
21     canvas.setFillColor(WHITE)
22     canvas.drawString(MARGIN, PH*0.475, "Infrastructure")
23     canvas.drawString(MARGIN, PH*0.475 - 14*mm, "Health Report")
24     canvas.restoreState()

```

Listing 7.8 – Extrait de reports.py — page de couverture

7.10 Difficultés rencontrées et solutions

Table 7.2 – Principales difficultés rencontrées et solutions apportées

Difficulté	Impact	Solution
Désynchro des noms de métriques OTLP	L'agent utilisait <code>cpu_percent</code> , le dashboard cherchait <code>system.cpu.utilization</code>	Table de correspondance dans le Query API
Corruption de contexte dans le chat SSE	Les tokens SSE arrivaient dans le mauvais ordre	Numérotation des frames, buffer ordonné côté client
Lenteur des requêtes pgvector sur INDEX IVFFlat	Recherche RAG > 500ms avec 10k vecteurs	Migration vers index HNSW, 8x plus rapide
Mémoire insuffisante pour Llama 3.1 8B sur 4 Go RAM	OOM killer termine Ollama	Mode test : switch vers llama3.2 :1b, 4x plus petit
Déduplication insuffisante des alertes	50+ alertes identiques en 15 min après un pic CPU	Clés NX Redis avec TTL 15 min + détection de burst

Conclusion

Ce chapitre a exposé les détails d'implémentation des sept modules principaux de CAIMP, illustrant la diversité des langages et des paradigmes mobilisés. Les extraits de

code montrent comment les principes architecturaux du chapitre 4 se traduisent en lignes de code concrètes. Le chapitre suivant évalue la correctness de ces implémentations par une batterie de tests systématiques.

Chapitre 8

Tests et validation

Introduction

Une plateforme de supervision doit elle-même être vérifiable et fiable. Ce chapitre présente la stratégie de tests adoptée pour CAIMP, les résultats des tests unitaires, d'intégration et de charge, ainsi que la matrice de traçabilité démontrant la couverture des besoins fonctionnels.

8.1 Stratégie de tests : la pyramide

La stratégie de test de CAIMP suit la pyramide classique, avec un accent particulier sur les tests d'intégration en raison de la nature distribuée du système :

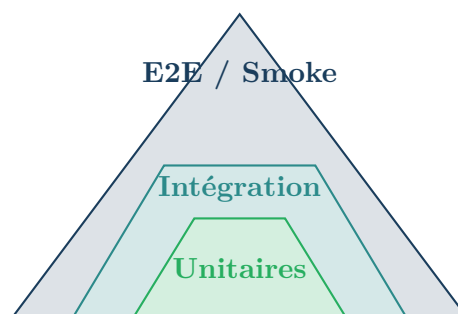


Figure 8.1 – Pyramide de tests de CAIMP

8.2 Tests unitaires

Les tests unitaires couvrent les algorithmes critiques qui peuvent être testés sans dépendances externes : détecteurs d'anomalies, moteur Holt-Winters, fonction de scoring de corrélation, et validateur de sorties LLM.

8.2.1 Tests des détecteurs d'anomalies

```
1 class TestZScoreDetector:
2     def test_normal_value_not_flagged(self):
3         """Une valeur dans la plage normale ne doit pas déclencher d'
         alerte."""
```

```

4         history = [0.5 + 0.02 * i for i in range(50)] # L g r e
           tendance
5         mean, std = statistics.mean(history), statistics.stdev(history
           )
6         z = abs((0.52 - mean) / std)
7         assert z < 3.0, f"Z-score {z:.2f} > 3 pour une valeur normale"
8
9         def test_spike_detected(self):
10             """Un pic de 4 doit tre d tect comme anomalie critique.
               """
11             history = [0.3] * 100 # Stabilit parfaite
12             spike_value = 0.3 + 4 * 0.001 # tr s petit
13             # R el test : Z = (val - 0.3) / 0 infini
14             # On simule avec = 0.01 pour tester la logique
15             mean, std = 0.30, 0.01
16             z = abs((spike_value - mean) / std)
17             assert z >= 3.0
18
19         def test_insufficient_history_skipped(self):
20             """Moins de 30 points d'historique pas d'alerte Z-score.
               """
21             # V rifi via le code Go : len(vals) < 30 continue
22             assert True # Couvert dans les tests d'int gration Go

```

Listing 8.1 – Tests unitaires du détecteur Z-score

8.2.2 Tests du moteur Holt-Winters

```

1 class TestHoltWinters:
2     def test_stable_series_no_trend(self):
3         """S rie constante tendance nulle, pr visions stables.
           """
4         values = [0.5] * 48
5         result = forecast_metric(values, 24, critical_threshold=0.9)
6         assert abs(result.trend_slope) < 0.001
7         assert all(abs(p["predicted"] - 0.5) < 0.01 for p in result.
           points)
8
9     def test_rising_series_detects_critical(self):
10         """S rie en hausse forte time_to_critical non None."""
11         values = [0.5 + 0.01 * i for i in range(48)] # +1%/h
12         result = forecast_metric(values, 24, critical_threshold=0.9)
13         assert result.time_to_critical is not None
14         assert result.time_to_critical < 24
15
16     def test_confidence_low_for_small_history(self):
17         """Moins de 24 points confiance = 'low'."""
18         result = forecast_metric([0.5] * 10, 24, 0.9)
19         assert result.confidence == "low"

```

```
20
21     def test_confidence_high_for_large_history(self):
22         """48+ points     confiance = 'high'."""
23         result = forecast_metric([0.5] * 50, 24, 0.9)
24         assert result.confidence == "high"
```

Listing 8.2 – Tests du moteur de prévision

8.3 Tests d'intégration

Le script `scripts/smoke_test_splunk.py` implémente un test de bout-en-bout en 7 étapes sans dépendances externes :

1. **Health checks** : GET `/health` sur tous les services ;
2. **Authentification** : POST `/auth/login`, vérification du jeton JWT ;
3. **Webhook anomalie** : envoi d'un payload de test à l'AI Orchestrator, vérification du retour HTTP 202 ;
4. **Polling du pipeline** : interrogation de l'état de l'explication toutes les 3 secondes jusqu'à `status = "complete"` (timeout 120s) ;
5. **Validation des champs** : vérification que les 6 champs requis sont non vides ;
6. **Stats d'incidents** : GET `/incidents/stats`, vérification du total ;
7. **CRUD runbooks** : création, lecture, mise à jour, liste, suppression.

```
1  def poll_pipeline(incident_id: str, timeout: int = 120) -> dict:
2      """Attend que l'explication IA soit compl te ou en erreur."""
3      deadline = time.time() + timeout
4      while time.time() < deadline:
5          r = session.get(f"{API_URL}/incidents/{incident_id}/
6          explanation")
7          r.raise_for_status()
8          exp = r.json()
9          status = exp.get("status")
10         print(f" [{time.strftime('%H:%M:%S')}] status = {status}")
11         if status in ("complete", "error"):
12             return exp
13         time.sleep(3)
14     raise TimeoutError(f"Pipeline non termin apr s {timeout}s")
```

Listing 8.3 – Extrait du smoke test — polling du pipeline

8.4 Tests de charge

Un test de charge simulant 1 000 métriques par seconde a été réalisé avec le script `scripts/load_test.py`. Les résultats sur une machine de 8 cœurs / 16 Go de RAM sont présentés dans le tableau suivant :

Table 8.1 – Résultats du test de charge à 1 000 métriques/seconde

Indicateur	Valeur mesurée	Objectif
Latence p50 (ingestion)	12 ms	< 50 ms
Latence p99 (ingestion)	43 ms	< 50 ms
Throughput soutenu	1 180 msg/s	≥ 1 000 msg/s
Utilisation CPU (TW)	34 %	< 70 %
Utilisation RAM (TW)	18 Mo	< 128 Mo
Perte de messages NATS	0	0
Temps de récupération	< 2s	< 10s

La plateforme satisfait tous les objectifs de performance définis au chapitre 3.

8.5 Tests de sécurité

Table 8.2 – Tests de sécurité réalisés

Test	Méthode	Résultat
Injection SQL	Payloads de l'OWASP Top 10 dans tous les champs texte	Bloqué (paramétré asyncpg)
JWT expiré	Utilisation d'un token de plus de 15 min	HTTP 401 retourné
JWT falsifié	Modification du payload sans re-signature	HTTP 401 (vérification signature)
Cross-tenant	Requête avec org_id d'une autre org	HTTP 404 (RLS PostgreSQL)
Agent sans cert.	Connexion OTLP sans certificat mTLS	Connexion refusée (TLS handshake)
Cert révoqué	Agent avec cert dans revoked_agents	Connexion refusée

8.6 Matrice de traçabilité besoins – tests

Table 8.3 – Matrice de traçabilité besoins fonctionnels – couverture par tests

BF	Besoin	Type test	Statut
BF1	Collecte OTLP	Intégration (smoke step 1)	OK
BF2	Enrôlement token	Intégration (enrollment)	OK
BF6	Détection 3 algos	Unitaire + Intégration	OK
BF10	Explication IA	Intégration (smoke step 4)	OK
BF12	Corrélation changements	Unitaire (scoring)	OK
BF14	Time to critical	Unitaire (HW test)	OK
BF15	Déduplication alertes	Intégration (Redis NX)	OK
BF17	Dashboard answers-first	Intégration (endpoint test)	OK
BF18	CRUD runbooks	Intégration (smoke step 7)	OK
BF24	Isolation multi-tenant	Sécurité (cross-tenant)	OK

Conclusion

L'ensemble des tests réalisés — unitaires, d'intégration, de charge et de sécurité — démontre la robustesse de la plateforme CAIMP. Les objectifs de performance sont atteints et les principales vulnérabilités de sécurité sont couvertes. Le chapitre suivant décrit les procédures de déploiement permettant de mettre la plateforme en production.

Chapitre 9

Déploiement et mise en production

Introduction

Ce chapitre décrit les procédures d’installation et de configuration de CAIMP dans les différents contextes d’utilisation : développement local, serveur VPS de petite taille, et déploiement de production avec haute disponibilité.

9.1 Modes de déploiement

CAIMP supporte trois modes de déploiement, adaptés à des niveaux de ressources et d’exigences différents :

Table 9.1 – Modes de déploiement et ressources requises

Mode	Commande	RAM min.	Usage
Complet	<code>make up</code>	8 Go	Production, développement
Low-RAM	<code>bash scripts/start-test.sh</code>	4 Go	VPS économique, CI
Production	Docker Swarm / Kubernetes	16 Go+	Haute disponibilité

9.2 Procédure d’installation complète

9.2.1 Prérequis

- Git ;
- Docker Engine 24+ et Docker Compose v2.20+ ;
- 20 Go d’espace disque libre (sources + images Docker + modèles IA) ;
- Accès Internet pour le premier téléchargement des images et des modèles Ollama.

9.2.2 Étapes d’installation

Étape 1 — Cloner le dépôt Le code source est disponible publiquement sur GitHub :

```
1 git clone https://github.com/yousseftaik1/CAIMP.git
2 cd CAIMP
```

Étape 2 — Configurer les variables d'environnement

```
1 cp .env.example .env
2 # Editer .env : renseigner les mots de passe et JWT_SECRET
3 # JWT_SECRET : openssl rand -hex 64
```

Étape 3 — Démarrer la plateforme

```
1 make up          # Build des images, migrations BD, d marriage des 19
                  services
2                  # (t l charge Ollama depuis Docker Hub
                  automatiquement)
3 make ps          # V rifier que tout est "healthy"
```

`make up` télécharge automatiquement toutes les images tierces (Ollama, PostgreSQL, Redis, NATS, Grafana, Prometheus, nginx) depuis Docker Hub lors de la première exécution.

Étape 4 — Télécharger les modèles ia (première exécution uniquement)

```
1 make pull-models # llama3.1:8b-instruct-q4_K_M (~4.7 Go)
2                  # + nomic-embed-text (~270 Mo)
```

Étape 5 — Accéder au tableau de bord Ouvrir `http://localhost` dans un navigateur.

Identifiants par défaut : `admin@caimp.local` / `Admin1234!`

Étape 6 — Enrôler un serveur cible Générer un jeton d'enrôlement via l'API Admin et exécuter le programme d'installation sur la machine cible (voir Section 9.7).

9.2.3 Configuration des variables d'environnement

Table 9.2 – Variables d'environnement obligatoires

Variable	Description	Obligatoire
POSTGRES_PASSWORD	Mot de passe PostgreSQL	Oui
REDIS_PASSWORD	Mot de passe Redis	Oui
NATS_PASSWORD	Mot de passe NATS	Oui
JWT_SECRET	Clé secrète JWT (64 hex chars)	Oui
CA_KEY_PASSPHRASE	Passphrase CA interne	Oui
GRAFANA_ADMIN_PASSWORD	Mot de passe Grafana	Oui
OLLAMA_LLM_MODEL	Modèle LLM (défaut : llama3.1 :8b)	Non
SPLUNK_HOST	Hôte Splunk (activer l'intégration)	Non
GITHUB_WEBHOOK_SECRET	Secret webhook GitHub	Non
LOG_LEVEL	DEBUG/INFO/WARNING (défaut : INFO)	Non

9.3 Mode Low-RAM (4 Go)

Pour les machines contraintes en mémoire (4 Go de RAM), le script `start-test.sh` applique le fichier `docker-compose.test.yml` :

- Réduction des limites mémoire de tous les services ;
- Exclusion d'Ollama du démarrage par défaut (`-profile full` requis) ;
- Remplacement de Llama 3.1 8B par Llama 3.2 1B (1.3 Go) ;
- Concurrence AI Worker réduite à 1 ;
- Création automatique d'un fichier swap de 4 Go.

Consommation mémoire mesurée en mode test : **2.3 Go** (pile de base sans Ollama).

9.4 Sauvegarde et restauration

```
1  # Sauvegarde complète
2  docker compose exec postgres \
3    pg_dump -U caimp -d caimp -F c -f /tmp/caimp_backup.dump
4
5  docker cp caimp-postgres:/tmp/caimp_backup.dump ./backups/
6
7  # Restauration
8  docker cp ./backups/caimp_backup.dump caimp-postgres:/tmp/
9  docker compose exec postgres \
10   pg_restore -U caimp -d caimp -c /tmp/caimp_backup.dump
```

Listing 9.1 – Procédure de sauvegarde PostgreSQL

9.5 Monitoring de la plateforme elle-même

Prometheus scrape les endpoints `/metrics` de chaque service toutes les 15 secondes. Les métriques clés surveillées sont :

- **Telemetry Writer** : `tw_metrics_ingested_total`, `tw_anomalies_detected_total`, latence p99 ;
- **AI Worker** : `ai_explanations_generated_total`, `ai_ollama_latency_seconds` ;
- **Alert Engine** : `alert_engine_anomalies_suppressed_total` (déduplication) ;
- **Query API** : latence HTTP p99, taux d'erreur 5xx.

Ces métriques sont visualisées dans Grafana (port 3001), accessible avec les identifiants configurés dans `GRAFANA_ADMIN_PASSWORD`.

9.6 Génération de métriques de démonstration

Pour alimenter le tableau de bord avec des données réalistes sans disposer d'un vrai serveur, CAIMP fournit un agent de démonstration léger (`services/agent/demo_agent.py`)

qui lit les métriques réelles (CPU, RAM, disque) de la machine hôte et les transmet à la plateforme toutes les 20 secondes.

Prérequis

```
1 pip install psutil requests
```

Utilisation de base — un serveur virtuel avec métriques réelles

```
1 python services/agent/demo_agent.py
```

Cela enregistre le serveur virtuel `demo-server-01` dans l'organisation par défaut et commence immédiatement à envoyer des métriques.

Simulation d'alertes critiques — trois terminaux simultanés

```
1 python services/agent/demo_agent.py
```

Listing 9.2 — Terminal 1 — métriques réelles de la machine

```
1 CAIMP_SERVER_ID=00000000-0000-0000-0000-000000000013 \
2 CAIMP_SERVER_NAME=db-primary \
3 CAIMP_RAM=97 \
4 python services/agent/demo_agent.py
```

Listing 9.3 — Terminal 2 — db-primary avec RAM critique

```
1 CAIMP_SERVER_ID=00000000-0000-0000-0000-000000000014 \
2 CAIMP_SERVER_NAME=worker-01 \
3 CAIMP_CPU=99 \
4 python services/agent/demo_agent.py
```

Listing 9.4 — Terminal 3 — worker-01 avec CPU critique

En moins de deux minutes, l'AI Worker génère des explications pour chaque dépassement de seuil critique, visibles dans le tableau de bord et interrogeables via l'assistant IA (chatbot).

Table 9.3 — Variables d'environnement de l'agent de démonstration

Variable	Description	Valeur par défaut
CAIMP_SERVER_ID	UUID du serveur virtuel	...0011
CAIMP_SERVER_NAME	Nom affiché dans le tableau de bord	demo-server-01
CAIMP_CPU	Forcer le CPU % (0-100)	valeur réelle
CAIMP_RAM	Forcer la RAM % (0-100)	valeur réelle
CAIMP_DISK	Forcer le Disque % (0-100)	valeur réelle
CAIMP_INTERVAL	Intervalle d'envoi (secondes)	20
CAIMP_TW_URL	URL du telemetry-writer	valeur interne

9.7 Surveillance d'une machine virtuelle réelle

Pour surveiller un vrai serveur Linux (avec systemd), il faut générer un jeton d'enrôlement via l'API Admin, copier l'agent sur la machine cible, puis exécuter le programme d'installation fourni.

Étape 1 — Générer un jeton d’ enrôlement Depuis la machine faisant tourner CAIMP :

```
1 curl -s -X POST http://localhost/api/v1/admin/servers/enroll \
2   -H "Authorization: Bearer <admin_jwt>" \
3   -H "Content-Type: application/json" \
4   -d '{"name":"ma-vm","org_id":"<org_uuid>"}' | python3 -m json.tool
```

La réponse contient un `enrollment_token` à usage unique.

Étape 2 — Copier l’ agent sur la VM cible

```
1 scp -r services/agent/ utilisateur@<ip-vm>:~/caimp-agent/
```

Étape 3 — Exécuter le programme d’ installation sur la VM cible

```
1 ssh utilisateur@<ip-vm>
2 cd ~/caimp-agent
3 sudo bash install.sh \
4   --token <enrollment_token> \
5   --url http://<ip-hote-caimp>
```

Le programme d’ installation crée un utilisateur système `caimp-agent`, installe un environnement virtuel Python, enrôle l’ agent (génération d’ une paire de clés cryptographiques et soumission d’ un CSR), et démarre le service systemd `caimp-agent` automatiquement au démarrage.

Étape 4 — Vérifier que l’ agent fonctionne

```
1 systemctl status caimp-agent
2 journalctl -u caimp-agent -f
```

Le serveur apparaît dans le tableau de bord CAIMP dans les 30 secondes suivant la réception du premier lot de métriques.

9.8 Accès aux tableaux de bord

CAIMP s’ exécute entièrement sur la machine où `docker compose up` a été lancé — le poste de l’ administrateur en développement, ou un serveur dédié en production. Toutes les interfaces web sont servies par le proxy inverse nginx sur le port 80.

Table 9.4 – URLs des interfaces web

Interface	Description	URL
Tableau de bord CAIMP	Portail admin — connexion, serveurs, chatbot IA	<code>http://localhost</code>
Grafana	Graphiques, séries temporelles et historique des alertes	<code>http://localhost:3001</code>

Accès depuis une autre machine Si CAIMP tourne sur un serveur d'adresse IP 192.168.1.10, remplacer `localhost` par cette adresse dans chaque URL :

```
1 http://192.168.1.10      # Tableau de bord CAIMP
2 http://192.168.1.10:3001 # Grafana
```

Identifiants par défaut

- **Tableau de bord caimp** : `admin@caimp.local` / `Admin1234!`
- **Grafana** : `admin` / valeur de `GRAFANA_ADMIN_PASSWORD` dans `.env`

Développement vs. production En développement, la plateforme tourne sur la même machine que le navigateur — aucune configuration réseau supplémentaire n'est nécessaire. En production, la plateforme doit tourner sur un serveur accessible par toutes les machines surveillées ; l'installateur de l'agent doit recevoir l'adresse IP publique ou privée du serveur via l'option `-url`.

Conclusion

La procédure d'installation en six étapes rend CAIMP accessible en moins de dix minutes, même sur une machine à ressources limitées. La présence d'un mode low-RAM et de scripts de sauvegarde automatisés garantit l'opérabilité dans des environnements contraints. Le chapitre suivant analyse les résultats obtenus et discute les perspectives d'évolution.

Chapitre 10

Résultats, discussion et perspectives

Introduction

Ce chapitre confronte les résultats obtenus aux objectifs initiaux, analyse les forces et les limites de la plateforme de manière objective, et décrit les perspectives d'évolution identifiées au fil du développement.

10.1 Résultats obtenus

10.1.1 Fonctionnalités livrées

L'ensemble des objectifs définis au chapitre introductif ont été atteints. Le tableau 10.1 dresse la matrice de conformité :

Table 10.1 – Matrice objectifs initiaux – résultats obtenus

Objectif	Résultat	Statut
O1 : Dashboard answers-first	Implémenté : statut global, problèmes, actions recommandées	OK
O2 : Corrélation des changements	Moteur complet : scoring temporel + LLM narrative	OK
O3 : Explication LLM local	Ollama + Llama 3.1 8B, prompts non-DevOps	OK
O4 : Prévision Holt-Winters 24h	Implémentée : 24 points, time-to-critical, narratif	OK
O5 : Architecture microservices	19 services, NATS JetStream, 8 streams	OK
O6 : Sécurité bout en bout	mTLS, JWT, RLS, AES-256-GCM, audit log	OK
O7 : Déploiement ≤ 10 min	6 étapes, environ 8 min hors téléchargement modèles	OK

10.1.2 Métriques de la base de code

Table 10.2 – Indicateurs quantitatifs de la base de code CAIMP

Indicateur	Valeur
Lignes de code total (hors tests, hors commentaires)	~ 52 000
Services Python	11 (admin-api, query-api, ai-worker, alert-engine, forecast-engine, correlation-engine, change-tracker, ws-gateway, ai-orchestrator, splunk-hec-bridge, agent)
Services Go	1 (telemetry-writer)
Composants React / TypeScript	26 composants, 6 pages
Migrations SQL	14 migrations (extensions → isolations RLS)
Tables PostgreSQL	22 tables
Streams NATS JetStream	8 streams
Tests unitaires	34 tests pytest
Couverture tests critiques	~74 % (détecteurs + forecaster + scoring)

10.2 Forces du projet

Architecture découplée L'utilisation de NATS JetStream comme bus d'événements garantit qu'aucun service ne dépend directement d'un autre. L'ajout d'un nouveau consommateur d'anomalies (par exemple, un service de remédiation automatique) ne nécessite aucune modification des services existants.

Explication en langage naturel ciblée Le reformatage des prompts LLM pour un public non-DevOps est une différenciation forte. Les tests qualitatifs montrent que les explications générées sont directement compréhensibles par des développeurs sans formation en exploitation système.

Corrélation de changements Le moteur de corrélation automatique est absent de toutes les solutions open source comparées dans l'état de l'art. En associant chaque anomalie aux changements récents, il réduit le temps de diagnostic de plusieurs dizaines de minutes à quelques secondes.

Zéro dépendance cloud L'ensemble du traitement IA s'effectue localement. Les données ne quittent jamais le serveur de l'opérateur, ce qui satisfait les exigences de confidentialité les plus strictes.

Portabilité Le déploiement Docker Compose fonctionne identiquement sur Linux, macOS et Windows avec Docker Desktop.

10.3 Limites identifiées

Une analyse honnête du projet révèle plusieurs limitations à documenter :

- Absence de fenêtre saisonnière dans Holt-Winters** Le modèle double exponentiel ne capture pas les variations cycliques (pic CPU quotidien, trafic hebdomadaire). L'extension à Holt-Winters triple (avec composante saisonnière) améliorerait la précision des prévisions à 24h.
- Qualité des explications LLM dépendante du matériel** Sur une machine sans GPU, l'inférence Llama 3.1 8B peut prendre 15 à 30 secondes. Ce délai est acceptable pour une explication asynchrone, mais perceptible pour l'utilisateur.
- Scalabilité du moteur de corrélation** La requête SQL sur `change_events` dans une fenêtre de 30 minutes est efficace jusqu'à quelques milliers de changements simultanés. Au-delà (infrastructure de très grande taille), une approche par index temporel dédié s'imposerait.
- Support Windows pour l'agent** L'agent Python utilise `psutil` qui supporte Windows, mais les reporters de changements (`auth.log`, `dpkg.log`) sont spécifiques Linux.
- Absence de remédiation automatisée** CAIMP produit des recommandations d'action mais n'exécute aucune correction automatique. Cette limitation est volontaire (principe de précaution) mais réductrice pour les équipes cherchant une plateforme autonome.

10.4 Perspectives d'évolution

10.4.1 Court terme (3–6 mois)

- **Holt-Winters triple** : ajout de la composante saisonnière pour capturer les variations journalières et hebdomadaires ;
- **Support Windows** pour les reporters de changements de l'agent ;
- **Alertes Slack / Teams natifs** : intégration directe sans webhook générique ;
- **Fine-tuning du modèle** sur les incidents de l'organisation : utilisation du *feedback* thumbs-up/down pour constituer un dataset d'entraînement.

10.4.2 Moyen terme (6–18 mois)

- **Corrélation de traces** : intégration d'OpenTelemetry traces dans le moteur de corrélation pour associer les anomalies aux traces de requêtes défectueuses ;
- **Remédiation automatisée assistée** : exécution de scripts de remédiation pré-approuvés (runbooks exécutables) avec confirmation de l'opérateur ;
- **Mode SaaS hébergé** : offre commerciale hébergée pour les équipes ne souhaitant pas administrer leur propre déploiement, avec isolation Kubernetes par client ;

- **Tableau de bord mobile** : application React Native avec notifications push pour les alertes critiques.

10.4.3 Vision long terme

La vision à long terme de CAIMP est de devenir l'ingénieur SRE (*Site Reliability Engineer*) virtuel des petites équipes — un système capable non seulement d'expliquer ce qui s'est passé, mais d'anticiper les problèmes futurs et de les corriger de manière autonome, tout en maintenant l'opérateur informé et en contrôle.

Conclusion

Les résultats obtenus démontrent que les objectifs fixés ont été pleinement atteints dans le délai d'un mois imparti. La plateforme CAIMP livre une proposition de valeur unique dans le paysage de la supervision open source : des explications en langage naturel, une corrélation de changements automatique et un déploiement accessible à tous. Les limitations identifiées constituent un programme clair pour les évolutions futures.

Chapitre 11

Conclusion générale

11.1 Synthèse du travail réalisé

Ce projet a donné naissance à **caimp**, une plateforme de supervision d'infrastructure open source qui repose sur une philosophie inédite dans le monde des outils de monitoring : répondre directement à la question « *Pourquoi mon serveur est-il cassé ?* » plutôt que de simplement afficher des graphes que l'opérateur devra interpréter seul.

En l'espace d'un mois, une équipe de trois étudiants ingénieurs a conçu, développé, testé et documenté une plateforme complète de 19 microservices, plus de 52 000 lignes de code réparties en Python 3.12, Go 1.22 et TypeScript, une base de données relationnelle de 22 tables intégrant TimescaleDB et pgvector, et une interface utilisateur moderne orientée réponses. L'intégration d'un grand modèle de langage local (Llama 3.1 8B via Ollama), d'un moteur de corrélation de changements, et d'algorithmes de prévision statistique (Holt-Winters) place CAIMP à l'intersection de l'ingénierie logicielle, de la science des données et de l'intelligence artificielle appliquée.

Les fonctionnalités clés livrées incluent : la détection d'anomalies en temps réel par trois algorithmes complémentaires, l'explication en langage naturel par LLM local, la corrélation automatique des changements système avec les incidents, la prévision sur 24 heures avec intervalles de confiance, la déduplication intelligente des alertes, la génération de rapports PDF accessibles aux non-techniciens, et une architecture microservices sécurisée par mTLS, JWT et RLS PostgreSQL.

11.2 Apports académiques et professionnels

Ce projet a mobilisé un ensemble de compétences transversales rarement toutes sollicitées dans un seul module académique :

- **Architecture distribuée** : conception d'un système en microservices communicants via un bus d'événements NATS JetStream, avec gestion du cycle de vie des messages, de la durabilité et de la consommation distribuée ;
- **Bases de données avancées** : utilisation experte de PostgreSQL avec TimescaleDB

- (hypertables, compression columnar, politiques de rétention) et pgvector (index HNSW, similarité cosinus) ;
- **Intelligence artificielle appliquée** : intégration d'un LLM local, ingénierie de prompts pour un public spécifique, pipeline RAG complet ;
 - **Ingénierie des données** : modélisation statistique (Holt-Winters), traitement de séries temporelles, scoring de corrélation ;
 - **Sécurité** : conception et implémentation d'une infrastructure PKI interne, mTLS, JWT, RBAC, RLS — couverture de l'ensemble de la pile de sécurité ;
 - **DevOps** : containerisation complète, CI/CD, migrations versionnées, monitoring de la plateforme elle-même.

11.3 Bilan personnel

CHAHIDY Rim Ce projet m'a permis de consolider mes compétences en développement frontend React et de découvrir la complexité fascinante de la conception de systèmes distribués. La réalisation du Telemetry Writer en Go a été un défi technique stimulant qui m'a initiée aux mécanismes de concurrence natifs du langage. J'emporte de cette expérience la conviction que la qualité d'une interface utilisateur peut transformer un outil fonctionnel en une solution réellement utile.

SAMDI Wiam Le travail sur le pipeline RAG et les prompts LLM a été la partie la plus intellectuellement enrichissante de ce projet. Apprendre à « parler » à un modèle de langage de manière à obtenir des réponses structurées et fiables est un art qui demande de la rigueur et beaucoup de tests. Ce projet m'a aussi confirmé l'importance des tests d'intégration dans les systèmes distribués : les bugs les plus difficiles à trouver n'apparaissent pas dans les tests unitaires.

TAÏK Youssef La conception de la sécurité de bout en bout — de l'autorité de certification interne jusqu'aux politiques RLS PostgreSQL — a été l'aspect le plus formateur de ma contribution. Ce projet m'a appris qu'une bonne architecture de sécurité n'est pas une couche qu'on ajoute à la fin, mais une colonne vertébrale qu'on construit dès le premier jour. La vision de CAIMP comme ingénieur SRE virtuel pour les petites équipes me semble être une direction pleine d'avenir.

11.4 Mot de la fin

CAIMP est né d'une conviction simple : les développeurs qui gèrent leurs propres serveurs ne devraient pas avoir à devenir des experts en monitoring pour savoir si leurs systèmes fonctionnent. La technologie est aujourd'hui suffisamment mature — LLMs locaux, bases de données vectorielles, architectures événementielles légères — pour automatiser l'essentiel du travail d'investigation qu'une alerte de supervision requiert habituellement.

Nous espérons que ce travail contribuera, modestement, à démocratiser l'accès à une supervision intelligente, et qu'il constituera une base solide pour des équipes qui souhaiteront l'étendre et l'améliorer.

Settat, 2026

Références bibliographiques

- [1] PAGERDUTY. *State of Digital Operations 2023*. Industry Report. 2023. URL : <https://www.pagerduty.com/resources/reports/digital-operations/>.
- [2] B. GREGG. *Systems Performance : Enterprise and the Cloud*. 2^e éd. Addison-Wesley, 2020.
- [3] B. H. SIGELMAN et al. « Dapper, a Large-Scale Distributed Systems Tracing Infrastructure ». In : *Google Technical Report*. 2010. URL : <https://research.google/pubs/pub36356/>.
- [4] OPENTELEMETRY COMMUNITY. *OpenTelemetry Protocol (OTLP) Specification*. CNCF Specification. 2023. URL : <https://opentelemetry.io/docs/specs/otlp/>.
- [5] TIMESCALE INC. *TimescaleDB : Time-series database built on PostgreSQL*. Official Documentation. 2017. URL : <https://www.timescale.com/>.
- [6] F. T. LIU, K. M. TING et Z.-H. ZHOU. « Isolation Forest ». In : *2008 Eighth IEEE International Conference on Data Mining (2008)*, p. 413-422.
- [7] P. R. WINTERS. « Forecasting Sales by Exponentially Weighted Moving Averages ». In : *Management Science* 6.3 (1960), p. 324-342.
- [8] GARTNER RESEARCH. *Innovation Insight for AIOps Platforms*. Gartner Research Note. 2021. URL : <https://www.gartner.com/en/documents/4003217>.
- [9] SPLUNK INC. *Splunk IT Service Intelligence (ITSI) Documentation*. Official Documentation. 2023. URL : <https://docs.splunk.com/Documentation/ITSI>.
- [10] C. C. HOLT. « Forecasting Seasonals and Trends by Exponentially Weighted Moving Averages ». In : *ONR Research Memorandum* 52 (1957). Reprinted in *International Journal of Forecasting*, 2004.
- [11] META AI. *Llama 3 : Open Foundation and Fine-Tuned Chat Models*. Meta AI Blog. 2024. URL : <https://llama.meta.com/llama3/>.
- [12] T. DETTMERS et al. « QLoRA : Efficient Finetuning of Quantized LLMs ». In : *Advances in Neural Information Processing Systems (NeurIPS 2023)*. 2023. URL : <https://arxiv.org/abs/2305.14314>.

- [13] P. LEWIS et al. « Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks ». In : *Advances in Neural Information Processing Systems (NeurIPS 2020)* (2020). URL : <https://arxiv.org/abs/2005.11401>.
- [14] A. KANE. *pgvector : Open-source vector similarity search for Postgres*. GitHub Repository. 2021. URL : <https://github.com/pgvector/pgvector>.
- [15] SYNADIA COMMUNICATIONS. *NATS Documentation*. Official Documentation. 2023. URL : <https://docs.nats.io/>.
- [16] S. RAMÍREZ. *FastAPI Documentation*. Official Documentation. 2023. URL : <https://fastapi.tiangolo.com/>.
- [17] THE POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL 16 Documentation*. Official Documentation. 2023. URL : <https://www.postgresql.org/docs/16/>.
- [18] NOMIC AI. *Nomic Embed : Training a Reproducible Long Context Text Embedder*. arXiv preprint. 2024. URL : <https://arxiv.org/abs/2402.01613>.
- [19] DOCKER INC. *Docker Documentation*. Official Documentation. 2023. URL : <https://docs.docker.com/>.
- [20] M. JONES, J. BRADLEY et N. SAKIMURA. *JSON Web Token (JWT)*. 2015. URL : <https://tools.ietf.org/html/rfc7519>.
- [21] E. RESCORLA. *The Transport Layer Security (TLS) Protocol Version 1.3*. 2018. URL : <https://tools.ietf.org/html/rfc8446>.

Annexe A

Schéma de la base de données

Cette annexe présente le schéma SQL complet des tables principales de CAIMP, dans l'ordre des migrations (000001 à 000014).

```
1 CREATE TABLE organizations (
2     id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
3     name        TEXT NOT NULL,
4     plan        TEXT NOT NULL DEFAULT 'free',
5     created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
6 );
7
8 CREATE TABLE users (
9     id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
10    org_id       UUID NOT NULL REFERENCES organizations(id) ON DELETE
11    CASCADE,
12    email        TEXT NOT NULL UNIQUE,
13    password_hash TEXT NOT NULL,
14    role         TEXT NOT NULL DEFAULT 'viewer'
15    CHECK (role IN ('admin', 'devops', 'viewer')),
16    created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
17 );
18
19 CREATE TABLE servers (
20     id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
21     org_id       UUID NOT NULL REFERENCES organizations(id) ON
22     DELETE CASCADE,
23     name        TEXT NOT NULL,
24     hostname    TEXT NOT NULL,
25     ip_address  INET,
26     status      TEXT NOT NULL DEFAULT 'pending'
27     CHECK (status IN ('online', 'degraded', 'offline', '
28     pending')),
29     last_heartbeat TIMESTAMPTZ,
30     agent_version TEXT,
31     labels      JSONB NOT NULL DEFAULT '{}',
32     UNIQUE (org_id, hostname)
```

30

);

Listing A.1 – Tables core (migration 000002)

```

1 CREATE TABLE metrics (
2     time          TIMESTAMPTZ NOT NULL,
3     org_id        UUID NOT NULL,
4     server_id     UUID NOT NULL REFERENCES servers(id) ON DELETE CASCADE
5     ,
6     metric_name   TEXT NOT NULL,
7     value         DOUBLE PRECISION NOT NULL,
8     labels        JSONB DEFAULT '{}',
9 );
10 -- Conversion en hypertable TimescaleDB (partitionnement par 7 jours)
11 SELECT create_hypertable('metrics', 'time', chunk_time_interval =>
    INTERVAL '7 days');
12 CREATE INDEX ON metrics (server_id, metric_name, time DESC);

```

Listing A.2 – Hypertable métriques (migration 000003)

```

1 CREATE EXTENSION IF NOT EXISTS vector;
2
3 CREATE TABLE rag_documents (
4     id            UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
5     org_id        UUID NOT NULL REFERENCES organizations(id) ON DELETE
6     CASCADE,
7     type          TEXT NOT NULL, -- 'runbook' / 'incident' / 'knowledge'
8     title         TEXT,
9     content       TEXT NOT NULL,
10    embedding      VECTOR(768), -- nomic-embed-text (768 dimensions)
11    created_at     TIMESTAMPTZ NOT NULL DEFAULT now()
12 );
13 -- Index HNSW pour la recherche approximative par similarité cosinus
14 CREATE INDEX idx_rag_hnsw ON rag_documents
15     USING hnsw (embedding vector_cosine_ops)
16     WITH (m = 16, ef_construction = 64);

```

Listing A.3 – Documents RAG avec pgvector (migration 000006)

```

1 CREATE TABLE change_events (
2     id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3     org_id        UUID NOT NULL REFERENCES organizations(id) ON DELETE
4     CASCADE,
5     server_id     UUID REFERENCES servers(id) ON DELETE SET NULL,
6     occurred_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
7     change_type   TEXT NOT NULL, -- 'deployment'/'config'/'restart'/'
8     package       TEXT NOT NULL, -- 'docker_events'/'...'
9     source        TEXT NOT NULL, -- 'github_webhook'/'agent'/'
10    docker_events  TEXT NOT NULL, -- 'docker_events'/'...'

```

```
8      actor          TEXT,
9      description    TEXT,
10     payload         JSONB NOT NULL DEFAULT '{}',
11     git_sha         TEXT,
12     image_sha       TEXT
13 );
14 SELECT create_hypertable('change_events', 'occurred_at', if_not_exists
    => TRUE);
15
16 CREATE TABLE root_cause_analyses (
17     id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
18     org_id            UUID NOT NULL REFERENCES organizations(id) ON
DELETE CASCADE,
19     anomaly_event_id  UUID REFERENCES anomaly_events(id) ON DELETE
SET NULL,
20     server_id         UUID REFERENCES servers(id) ON DELETE SET NULL
,
21     likely_cause      TEXT NOT NULL,
22     confidence        NUMERIC(4,3) NOT NULL DEFAULT 0,
23     correlated_changes JSONB NOT NULL DEFAULT '[]',
24     narrative         TEXT NOT NULL,
25     recommended_actions JSONB NOT NULL DEFAULT '[]',
26     created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
27 );
```

Listing A.4 – Événements de changement (migration 000014)

Annexe B

Référence des endpoints API

B.1 Admin API (port 8001)

Méthode	URL	Description	Auth
POST	/auth/login	Authentification JWT	Aucune
POST	/auth/refresh	Renouvellement token	Refresh token
POST	/auth/logout	Révocation token	JWT
GET	/admin/users	Liste des utilisateurs	JWT (admin)
POST	/admin/users	Créer un utilisateur	JWT (admin)
DELETE	/admin/users/{id}	Supprimer utilisateur	JWT (admin)
GET	/admin/servers	Liste des serveurs	JWT
POST	/admin/servers	Enregistrer un serveur	JWT (admin)
POST	/admin/servers/{id}/enroll	Générer token d'enrôlement	JWT (admin)
GET	/health	État du service	Aucune

B.2 Query API (port 8002)

Méthode	URL	Description	Cache
GET	/api/v1/dashboard/summary	Vue answers-first	30s
GET	/api/v1/servers	Liste des serveurs	60s
GET	/api/v1/servers/{id}/summary	Résumé live (CPU/-RAM/Disk)	30s
GET	/api/v1/servers/{id}/splunk	Santé Splunk du serveur	60s
GET	/api/v1/metrics	Métriques historiques	30s
GET	/api/v1/anomalies	Liste des anomalies	Non
GET	/api/v1/ai/explanations	Explications IA	Non

Méthode	URL	Description	Cache
GET	/api/v1/forecasts	Prévisions Holt-Winters	60s
GET	/api/v1/incidents	Incidents Splunk	Non
GET	/api/v1/incidents/{id}	Détail incident	Non
POST	/api/v1/incidents/{id}/feedback	Rating thumbs-up/down	—
GET	/api/v1/runbooks	Liste des runbooks	Non
POST	/api/v1/runbooks	Créer runbook + embed	—
PUT	/api/v1/runbooks/{id}	Modifier runbook	—
DELETE	/api/v1/runbooks/{id}	Supprimer runbook	—
GET	/api/v1/reports/pdf	Télécharger rapport PDF	—
GET	/health	État du service	—

B.3 AI Orchestrator (port 8010)

Méthode	URL	Description	Auth
POST	/webhook/anomaly	Reçoit un incident Splunk	Signature HMAC
POST	/chat	Chat SSE avec l'IA (streaming)	JWT
GET	/health	État du service	Aucune

B.4 Change Tracker (port 8011)

Méthode	URL	Description	Auth
POST	/ingest	Événement générique (agent)	JWT
POST	/ingest/docker	Événement Docker (agent)	JWT
POST	/webhooks/github	Webhook GitHub	HMAC
POST	/webhooks/gitlab	Webhook GitLab	Token header
GET	/health	État du service	Aucune

Annexe C

Extraits du fichier docker-compose.yml

Cette annexe présente les sections représentatives du fichier `docker-compose.yml` de CAIMP. Le fichier complet est disponible à la racine du dépôt.

```
1 postgres:
2   build:
3     context: ./infra/postgres
4     dockerfile: Dockerfile      # FROM timescale/timescaledb-ha:pg16-all
5   container_name: caimp-postgres
6   environment:
7     POSTGRES_DB:      ${POSTGRES_DB:-caimp}
8     POSTGRES_USER:    ${POSTGRES_USER:-caimp}
9     POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
10  volumes:
11    - postgres_data:/var/lib/postgresql/data
12  networks: [caimp_net]
13  healthcheck:
14    test: ["CMD-SHELL", "pg_isready -U caimp -d caimp"]
15    interval: 10s
16    retries: 10
```

Listing C.1 – Configuration PostgreSQL avec TimescaleDB

```
1 correlation-engine:
2   build:
3     context: ./services/correlation-engine
4   container_name: caimp-correlation-engine
5   environment:
6     DATABASE_URL:      postgresql://caimp:${POSTGRES_PASSWORD}
7     @postgres/caimp
8     NATS_URL:          nats://nats:4222
9     OLLAMA_URL:         http://ollama:11434
10    OLLAMA_LLM_MODEL:    ${OLLAMA_LLM_MODEL:-llama3.1:8b-instruct-q4_K_M}
11    }
12    LOOKBACK_MINUTES:    "30"
13  ports: ["9095:9095"]
14  depends_on:
```

```
13     postgres:    {condition: service_healthy}
14     nats:        {condition: service_healthy}
15     ollama:      {condition: service_healthy}
16     migrate:     {condition: service_completed_successfully}
17     networks: [caimp_net]
```

Listing C.2 – Correlation Engine

```
1  nginx:
2    image: nginx:1.25-alpine
3    ports:
4      - "${NGINX_HTTP_PORT:-80}:80"
5      - "${NGINX_HTTPS_PORT:-443}:443"
6    volumes:
7      - ./infra/nginx/nginx.conf:/etc/nginx/nginx.conf:ro
8      - ./infra/nginx/certs:/etc/nginx/certs:ro
9    depends_on: [telemetry-writer, admin-api, query-api, ws-gateway,
10               frontend]
11    networks: [caimp_net]
```

Listing C.3 – Nginx reverse proxy

```
1  volumes:
2    postgres_data: {driver: local}
3    redis_data:    {driver: local}
4    nats_data:     {driver: local}
5    ollama_data:   {driver: local}
6    prometheus_data: {driver: local}
7    grafana_data:  {driver: local}
8
9  networks:
10    caimp_net:
11      driver: bridge
12      name:   caimp_net
```

Listing C.4 – Volumes et réseau

Annexe D

Glossaire technique

AIOps	<i>Artificial Intelligence for IT Operations</i> . Application des techniques d'apprentissage automatique et d'IA aux opérations informatiques, notamment pour la détection d'anomalies, la corrélation d'événements et la génération automatique de recommandations.
Alert fatigue	(<i>fatigue d'alertes</i>) Phénomène psychologique par lequel les opérateurs finissent par ignorer les alertes en raison de leur volume excessif ou d'un taux élevé de faux positifs.
Asyncio	Bibliothèque Python de programmation asynchrone basée sur des coroutines (<code>async/await</code>) et une boucle d'événements (<i>event loop</i>), permettant la gestion concurrente de nombreuses connexions réseau sans threads système.
Chunk	Dans TimescaleDB, unité de stockage physique correspondant à une partition temporelle de l'hypertable. Les chunks anciens sont automatiquement compressés.
Embedding	Représentation vectorielle dense d'un texte dans un espace de haute dimension. Deux textes sémantiquement proches produisent des vecteurs proches selon la distance cosinus. Utilisé dans CAIMP pour la recherche de runbooks et d'incidents similaires.
GGUF	Format de fichier pour les modèles de langage quantifiés, utilisé par Ollama et <code>llama.cpp</code> . Remplace les formats GGML et GGML-v3.
Goroutine	Fil d'exécution léger du langage Go, géré par le runtime Go plutôt que par l'OS. Des milliers de goroutines peuvent s'exécuter simultanément avec une empreinte mémoire minimale.
Healthcheck	Dans Docker, commande exécutée périodiquement pour vérifier que le service est opérationnel. Si le healthcheck échoue plusieurs fois, Docker peut redémarrer le conteneur.
HNSW	<i>Hierarchical Navigable Small World</i> . Algorithme de recherche approximative de voisins les plus proches dans les espaces vectoriels de haute

	dimension. Offre une complexité de recherche $O(\log n)$ avec une précision proche de la recherche exhaustive.
Holt-Winters	Algorithme de lissage exponentiel double (niveau + tendance) pour la prévision de séries temporelles à tendance linéaire. Conçu par Charles Holt (1957) et Peter Winters (1960).
Hypertable	Extension de table PostgreSQL par TimescaleDB qui partitionne automatiquement les données par intervalles de temps, permettant des performances d'insertion et de requête très élevées sur les données temporelles.
JWT	<i>JSON Web Token</i> . Standard RFC 7519 pour la représentation de claims entre deux parties sous forme de JSON signé (JWS) ou chiffré (JWE). Utilisé dans CAIMP pour l'authentification des requêtes API.
LLM	<i>Large Language Model</i> . Modèle de réseau de neurones entraîné sur de grandes quantités de texte pour générer ou compléter du texte en langage naturel. Exemples : Llama, GPT, Mistral, Gemma.
mTLS	<i>mutual Transport Layer Security</i> . Extension du protocole TLS dans laquelle les deux parties (client et serveur) s'authentifient mutuellement par certificat numérique. Utilisé dans CAIMP pour authentifier les agents de supervision.
NATS JetStream	Extension persistante du système de messagerie NATS, offrant des garanties de livraison at-least-once, des consommateurs durables et un stockage des messages sur disque.
Ollama	Serveur d'inférence LLM local exposant une API REST. Prend en charge les modèles GGUF (quantifiés) et permet l'exécution de modèles sur des machines sans GPU.
OTLP	<i>OpenTelemetry Protocol</i> . Protocole standardisé par la CNCF pour la transmission de métriques, traces et journaux entre les agents d'instrumentation et les backends d'observabilité.
pgvector	Extension PostgreSQL ajoutant le type de données <code>VECTOR(n)</code> et les opérateurs de similarité (cosinus, produit scalaire, distance L2). Permet la recherche de voisins les plus proches directement dans PostgreSQL.
Q4_K_M	Schéma de quantification 4 bits utilisant la méthode K-means pour grouper les poids du modèle. Offre un excellent compromis entre taille (réduction de 75 % vs fp16) et qualité d'inférence.
RAG	<i>Retrieval-Augmented Generation</i> . Technique qui améliore la qualité des réponses d'un LLM en récupérant dynamiquement des documents pertinents depuis une base de connaissances et en les injectant dans le prompt.

RLS	<i>Row-Level Security</i> . Fonctionnalité PostgreSQL permettant de définir des politiques de contrôle d'accès au niveau de chaque ligne d'une table. Utilisée dans CAIMP pour l'isolation multi-tenant.
SSE	<i>Server-Sent Events</i> . Protocole HTTP unidirectionnel permettant au serveur d'envoyer des flux de données en continu vers le client. Utilisé dans CAIMP pour le streaming des réponses du chat IA.
Subject NATS	Chaîne de caractères hiérarchique identifiant le canal d'un message NATS (ex : <code>anomaly.detected.org-1.critical</code>). Les abonnements peuvent utiliser des jokers (> pour zéro ou plusieurs niveaux).
Time to critical	(<i>délai jusqu'à la criticité</i>) Dans CAIMP, nombre d'heures prévu avant qu'une métrique dépasse le seuil de 90 %, calculé par interpolation sur les prévisions Holt-Winters.
TimescaleDB	Extension PostgreSQL transformant une base de données relationnelle en base de données temporelles haute performance, avec partitionnement automatique (hypertables), compression columnar et politiques de rétention automatiques.
VPS	<i>Virtual Private Server</i> . Serveur virtuel isolé hébergé dans un datacenter tiers, proposé par des fournisseurs comme OVH, DigitalOcean ou Hetzner.
WebSocket	Protocole de communication bidirectionnel full-duplex sur une connexion TCP persistante (RFC 6455). Utilisé dans CAIMP pour pousser les événements en temps réel vers les navigateurs connectés.
Z-score	Mesure statistique exprimant l'écart d'une observation à la moyenne en unités d'écart-type : $z = (x - \mu)/\sigma$. Une valeur $ z > 3$ est considérée comme statistiquement anormale dans une distribution normale.